INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences
Maidan Garhi, New Delhi – 110 068

Project Report

Course Code: **BCSP-064**

BCA – 6th Semester

Hotel Room Booking Website with Assistant Chatbot

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Computer Applications (BCA)

Submitted By

Name: **Nikhil P N**

Enrolment No: **2350526613**

Programme: BCA

Study Centre Code: _____

Regional Centre: **IGNOU Kolkata Regional Centre**

Address: _____

Under the Guidance of

Guide Name: _____

Designation: _____

Qualification: _____

Address: _____

Academic Year: **2025 – 2026**

Certificate of Originality

This is to certify that the project report entitled **"Hotel Room Booking Website with Assistant Chatbot"** submitted to **Indira Gandhi National Open University (IGNOU)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)** is an original piece of work carried out by **Mr. Nikhil P N**, bearing Enrolment Number **2350526613**, under the guidance of **Mr./Ms. _____**.

The matter embodied in this project report is a genuine work done by the student and has not been submitted in part or in full, whether to this University or to any other University or Institute, for the fulfilment of the requirement of any course of study.

It is further certified that the student has fulfilled all the conditions of eligibility for submitting the project report, the report has been examined, and I recommend the same for the award of the degree of BCA.

_____

Signature of the Student

Name: Nikhil P N
Enrolment No: 2350526613
Date: _____

_____

Signature of the Guide

Name: _____
Designation: _____
Address: _____
Date: _____

Acknowledgement

I would like to express my sincere gratitude to **Indira Gandhi National Open University (IGNOU)** for providing me with the opportunity to undertake this project as part of the BCA programme. The knowledge and skills gained during the course of study have been instrumental in the successful completion of this project.

I am deeply indebted to my project guide, **Mr./Ms. _____**, whose valuable guidance, constant encouragement, and constructive suggestions throughout the duration of this project enabled me to complete it successfully. Their expertise in the field of computer science and willingness to spare their valuable time has been immensely helpful.

I extend my heartfelt thanks to the **IGNOU Kolkata Regional Centre** and my Study Centre for their administrative support and for ensuring the smooth progression of my academic journey throughout the BCA programme.

I also wish to acknowledge my gratitude to all the faculty members of the School of Computer and Information Sciences, IGNOU, whose teachings in subjects like Database Management Systems, Internet Technology, Software Engineering, Object-Oriented Technology, and Web Programming have provided me with the theoretical foundation and practical skills necessary for this project.

I am also thankful to the open-source community, the developers of the Django framework, the Python programming language, and

the various libraries and tools (Razorpay, xhtml2pdf, Bootstrap) used in this project. Their freely available documentation, tutorials, and forums have been invaluable resources.

Last but not least, I express my deepest gratitude to my family and friends for their constant moral support and encouragement throughout the period of this project work. Their faith in me kept me motivated even during challenging phases of development and testing.

Nikhil P N
Enrolment No: 2350526613
BCA – 6th Semester
IGNOU Kolkata Regional Centre

Table of Contents

- Data Flow Diagrams (DFD Level-0 & Level-1)
- Entity-Relationship Diagram
- Data Dictionary

11.Design Document

- Modularization Details (8 Modules)
- Database Design and Normalization
- Procedural Design / Process Logic
- User Interface Design

12.Source Code (Key Modules)

- Django Models
- Booking with Overlap Detection
- Advanced Search API
- Chatbot Engine
- Payment Integration
- OTP-Based Password Reset

13.Testing

- Testing Strategy
- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing
- Debugging and Bug Fixing

14.Validations Performed

15.Input and Output Screens

16.Report Generation

17.Implementation of Security

1. Abstract

The **Hotel Room Booking Website with Assistant Chatbot** is a comprehensive, full-stack web application developed using **Python**, the **Django 6.0** web framework, and a **PostgreSQL** relational database. The system has been designed and developed to automate end-to-end hotel operations including room-type management, room inventory control, customer registration, online room booking with date-based availability checking, payment processing through the Razorpay payment gateway, staff administration, customer contact handling, email notifications, PDF report generation, and an intelligent assistant chatbot that can answer guest queries about rooms, bookings, staff, and hotel policies.

The project follows the **Software Development Life Cycle (SDLC)** and encompasses all the major phases: Requirements Analysis, System Design, Coding, Testing, and Documentation. It consists of eight functional modules spanning both the administrator backend and the customer-facing website, and exposes RESTful API endpoints for the chatbot and advanced search features. The application is fully responsive and works seamlessly across desktop computers, tablets, and mobile devices.

The administrator can manage room types, rooms, staff profiles, customer enquiries, bookings, payments, chatbot responses, and notifications from a secure backend interface. Customers can register, sign in, browse rooms by category, search with advanced filters (room type, price range, dates), book available rooms with automatic overlap detection, make secure online payments via

Razorpay, interact with the chatbot for instant assistance, submit contact enquiries, and reset their passwords using email-based OTP verification.

The system stores data in a normalized relational database with ten interconnected tables and enforces referential integrity through primary-key and foreign-key constraints. Efficient querying, input validation, CSRF protection, and session-based access control ensure a secure and responsive user experience. The project demonstrates the practical application of RDBMS concepts, internet technology, and AI-assisted user interaction in the hospitality domain.

Keywords: Hotel Room Booking, Django, Python, PostgreSQL, Chatbot, Razorpay, AJAX, Bootstrap, PDF Generation, Email OTP, REST API, SDLC, Software Engineering.

2. Introduction

2.1 Background of the Problem

The hospitality industry is one of the largest and fastest-growing sectors of the global economy. Hotels, resorts, and guest houses serve millions of travellers every day and face the critical challenge of efficiently managing room inventory, customer bookings, payments, and guest communication. In many small and medium-sized hotels, especially in developing countries, room booking and management are still done using manual methods such as paper-based registers, phone-based reservations, and disconnected spreadsheets. These traditional approaches have numerous disadvantages including the risk of duplicate bookings, inaccurate room availability records, delayed guest communication, data loss, and difficulty in generating financial and occupancy reports.

With the advent of the internet and web technologies, customers increasingly expect the ability to search for rooms, compare prices, check availability in real time, and make bookings and payments online without visiting the hotel in person or calling the front desk. Hotels that fail to provide an online presence with booking capabilities risk losing customers to competitors who do. Furthermore, the increasing use of chatbots and AI-powered assistants in customer service has raised guest expectations for instant, 24/7 responses to common questions about room

availability, pricing, check-in/check-out times, and hotel policies.

This project aims to address all of these challenges by building a web-based Hotel Room Booking System that serves both the hotel administration and the customer, providing a single, unified platform for all hotel operations.

2.2 Motivation for the Project

The motivation for undertaking this project stems from several observations and requirements:

1. **Academic Requirement:** The BCSP-064 project is a mandatory component of the BCA 6th-semester curriculum at IGNOU. It requires the development of a software application that demonstrates competence in programming, database management, software engineering, and internet technology—all of which are core subjects in the BCA programme.
2. **Real-World Relevance:** The hotel industry has a genuine need for affordable, customizable booking solutions. Many small hotels cannot afford enterprise-level Property Management Systems (PMS) and would benefit from an open-source, Django-based system that can be deployed on a standard web server.
3. **Full-Stack Skill Development:** Developing this project provides hands-on experience with front-end technologies (HTML5, CSS3, Bootstrap 5, JavaScript), back-end development (Python, Django), database design (PostgreSQL, SQL), payment gateway integration (Razorpay), email services (Django SMTP), PDF generation (xhtml2pdf), and basic AI/chatbot development.

4. **Interest in AI-Assisted Services:** The inclusion of a chatbot module was motivated by the growing importance of conversational AI in customer service. Even a rule-based chatbot can significantly reduce the workload on hotel staff by answering frequently asked questions instantly.

2.3 Scope of the Project

The scope of this project covers the following functional areas:

- **Customer-Facing Website:** A responsive, publicly accessible website where customers can browse room categories, view room details and images, search rooms with advanced filters, register and sign in, make bookings, process payments, submit contact enquiries, interact with the chatbot, and reset their passwords via email OTP.
- **Administrator Backend:** A secure, login-protected admin panel where hotel staff can manage room types, rooms, staff profiles, view and delete customer contact messages, view booking records, view payment details, and manage chatbot responses and notifications via the Django Admin interface.
- **Database:** A normalized PostgreSQL relational database with ten tables covering room types, rooms, users, bookings, payments, staff, customer contacts, chatbot conversations, chatbot predefined responses, and notifications.
- **API Endpoints:** RESTful JSON API endpoints for the chatbot (query and rating), advanced room search, notification retrieval, and notification status updates.
- **Reporting:** On-screen reports for bookings, payments, contacts, and staff, as well as PDF download capability for booking records.

The project does *not* cover multi-hotel/multi-branch management, dynamic pricing, loyalty programmes, third-party OTA (Online Travel Agency) channel integration, or deployment on cloud infrastructure—these are identified as future enhancements.

3. Objectives of the Project

The primary objectives of this project are:

1. To design and develop a web-based hotel room booking system that automates room type management, room inventory management, booking management, staff management, and customer contact management using the Django web framework and Python programming language.
2. To provide customers with a user-friendly, responsive web interface for browsing hotel room categories, viewing room details (with images, prices, and descriptions), and searching for rooms using advanced filters including room name, room type, price range, and check-in/check-out date availability.
3. To implement date-range-based availability checking logic that prevents double booking of the same room by detecting overlapping check-in and check-out dates before confirming any booking.
4. To integrate an online payment gateway (Razorpay) for secure transaction processing so that customers can pay for their bookings online using credit cards, debit cards, UPI, and net banking.
5. To build an intelligent assistant chatbot that can classify user queries into categories (room, staff, booking, general) and provide relevant, context-aware responses including room cards with images and prices for room-related queries.
6. To implement advanced room search functionality with

real-time AJAX-based filtering that returns results without full page reload, enhancing the user experience.

7. To incorporate email notification services for booking confirmations and OTP-based password recovery so that customers receive immediate communication for critical actions.
8. To generate downloadable PDF reports of booking details using the xhtml2pdf library for record-keeping and future reference.
9. To maintain secure access control through user authentication, session management, CSRF protection, and input validation to prevent unauthorized access and common web security vulnerabilities.
10. To follow Software Engineering practices and the Software Development Life Cycle (SDLC) throughout the development process, including requirements gathering, system analysis, design, coding, testing, and documentation.

4. Project Category

This project falls under the following categories as defined by the IGNOU BCSP-064 guidelines:

Category	Description
RDBMS (Relational Database Management System)	The project uses a PostgreSQL relational database with ten normalized tables connected by primary-key and foreign-key relationships. The database design follows normalization principles (up to 3NF) to eliminate data redundancy and ensure data integrity. All data operations are performed through Django's Object-Relational Mapper (ORM), which translates Python model definitions into SQL queries.
Internet Technology / Web Application	The project is a full-stack web application built with the Django web framework. The frontend uses HTML5, CSS3, Bootstrap 5.3, and vanilla JavaScript (ES6). It employs AJAX

	(XMLHttpRequest / Fetch API) for asynchronous operations such as the advanced room search and chatbot communication. RESTful API endpoints return JSON responses. The application is accessible through any modern web browser over HTTP/HTTPS.
AI Application (Chatbot)	The project includes an assistant chatbot module that uses rule-based keyword classification to understand user intent and generate contextually relevant responses. The chatbot can query the database for rooms, staff, and bookings and present results in a conversational format with rich UI elements (room cards, suggestion chips). Conversations are stored for analytics, and users can rate responses for quality feedback.

5. System Study

5.1 Study of the Existing System

Before developing the proposed system, a thorough study of the existing hotel management practices was conducted. In many small and medium-sized hotels, the following manual and semi-automated systems are in use:

1. **Paper-Based Booking Register:** Room bookings are recorded manually in a physical register or diary. The receptionist writes the guest name, contact details, room number, check-in date, check-out date, and amount in the register. This register is the primary source of booking information.
2. **Phone-Based Reservations:** Customers call the hotel to enquire about room availability and make reservations. The receptionist checks the register manually, informs the caller about availability, and records the booking on paper or in a simple spreadsheet.
3. **Spreadsheet-Based Records:** Some hotels use Microsoft Excel or Google Sheets to maintain booking records, guest information, and payment details. While this is an improvement over paper registers, it still requires manual data entry, is prone to errors, and does not provide real-time availability checking.
4. **No Online Presence:** Many small hotels do not have a

website or online booking capability. Customers must visit the hotel in person or call to check availability and make bookings, which is inconvenient and limits the hotel's reach.

5. **No Automated Communication:** Booking confirmations, reminders, and other communications are sent manually via phone calls or occasionally by email. There is no automated system for sending confirmation emails, OTPs, or notifications.

5.2 Limitations of the Existing System

1. **Double Booking Risk:** Without real-time availability checking, two or more guests may be booked for the same room on overlapping dates, leading to embarrassing situations and guest dissatisfaction.
2. **Data Redundancy:** Guest information, room details, and booking records may be duplicated across multiple registers, spreadsheets, or files, leading to inconsistencies.
3. **Slow Retrieval:** Searching for a specific booking, guest record, or payment history in a paper register or large spreadsheet is time-consuming and error-prone.
4. **No Report Generation:** Generating summary reports for occupancy rates, revenue, payment collections, or guest demographics is extremely difficult and labour-intensive with manual systems.
5. **Security Concerns:** Paper registers can be lost, damaged, or accessed by unauthorized persons. Spreadsheets may not have proper access controls.
6. **No Online Access:** Customers cannot check room availability or make bookings from their homes or offices,

limiting the hotel's customer base.

7. **No 24/7 Assistance:** Without a chatbot or automated FAQ system, guest enquiries can only be answered during front-desk working hours.
8. **Delayed Payment Processing:** Payments are typically collected at check-in or check-out in cash. There is no facility for advance online payment, which affects the hotel's cash flow and booking commitment.

5.3 Proposed System

The proposed system is a web-based Hotel Room Booking application that addresses all the limitations of the existing system. It provides:

- A centralized relational database that stores all hotel data (room types, rooms, staff, bookings, payments, contacts, chatbot interactions) in normalized tables with referential integrity.
- An administrator backend where hotel staff can perform CRUD (Create, Read, Update, Delete) operations on room types, rooms, staff profiles, and can view booking records, payment totals, and customer messages.
- A customer-facing website where guests can register, sign in, browse rooms, search with advanced filters, book rooms with automatic date-overlap detection, make online payments via Razorpay, submit contact enquiries, and interact with a chatbot.
- Automated email notifications for booking confirmations and OTP-based password resets.

- PDF report generation for booking records.
- A responsive, mobile-friendly design using Bootstrap 5.3.

5.4 Advantages of the Proposed System

1. **Elimination of Double Booking:** The system checks for date-range overlaps before confirming any booking, ensuring that no room is booked twice for the same period.
2. **Centralized Data Management:** All data is stored in a single database, eliminating redundancy and ensuring consistency.
3. **Instant Search and Retrieval:** AJAX-powered advanced search allows staff and customers to find rooms, bookings, and records in milliseconds.
4. **Automated Reporting:** Booking summaries, payment totals, and contact records are generated automatically and can be exported as PDF.
5. **Enhanced Security:** Session-based authentication, CSRF protection, ORM-based parameterised queries, and input validation protect against unauthorized access and common web attacks.
6. **24/7 Online Access:** Customers can browse, search, and book rooms at any time from any device with an internet connection.
7. **Instant Customer Support:** The chatbot provides immediate responses to common queries about rooms, staff, bookings, and hotel policies, reducing the workload on front-desk staff.
8. **Secure Online Payments:** Integration with Razorpay allows customers to pay online using multiple payment methods,

improving cash flow and booking commitment.

9. **Scalability:** The modular Django architecture allows new modules to be added with minimal changes to the existing codebase.

6. Feasibility Study

A feasibility study was conducted before starting the development to assess whether the project is practical and viable. The study covered three dimensions: technical, operational, and economic feasibility.

6.1 Technical Feasibility

Technical feasibility examines whether the required technology, tools, and skills are available to develop the system.

- **Programming Language:** Python 3.x is a mature, widely-used, open-source language with extensive library support. It is well-suited for web development and AI applications.
- **Web Framework:** Django 6.0 is one of the most popular Python web frameworks. It follows the MVT (Model-View-Template) architecture, provides a built-in admin interface, ORM, form handling, authentication, CSRF protection, and URL routing, making it ideal for rapid application development.
- **Database:** PostgreSQL is used as the main relational database. It supports SQL standards, advanced features, and is suitable for both small and large-scale applications. The project is fully compatible with PostgreSQL for production deployment.
- **Frontend Technologies:** HTML5, CSS3, Bootstrap 5.3, and

JavaScript (ES6) are standard, well-documented, and supported by all modern browsers.

- **Payment Gateway:** Razorpay provides a well-documented Python SDK and test mode (sandbox) for development, making integration straightforward.
- **PDF Generation:** xhtml2pdf converts HTML templates to PDF documents, leveraging the same Django template engine used for web pages.
- **Development Tools:** VS Code (editor), Git (version control), and Chrome DevTools (debugging) are freely available and widely used.
- **Developer Skills:** The developer (student) has completed courses in Python, DBMS, Internet Technology, Software Engineering, and Web Programming as part of the BCA curriculum, providing the necessary skills.

Conclusion: The project is technically feasible. All required technologies are open-source, well-documented, and within the developer's skill set.

6.2 Operational Feasibility

Operational feasibility examines whether the system can be practically used in a real hotel environment.

- **Ease of Use:** The admin backend provides simple forms for adding, editing, and deleting data. The customer website uses an intuitive layout with clear navigation. No technical expertise is required to operate the system.
- **Training:** Minimal training is needed. The admin interface uses standard form controls (text fields, dropdowns, file

uploads, buttons). The customer website follows common e-commerce patterns that users are already familiar with.

- **Acceptance:** The system offers significant improvements over manual methods (speed, accuracy, convenience, 24/7 access), which should lead to high user acceptance.
- **Support:** The chatbot provides instant, automated customer support, reducing the dependency on human staff for routine enquiries.

Conclusion: The project is operationally feasible. It is user-friendly, requires minimal training, and offers clear benefits over existing manual processes.

6.3 Economic Feasibility

Economic feasibility assesses the cost-benefit ratio of the project.

Cost Component	Estimated Cost
Python, Django, PostgreSQL, Bootstrap, JavaScript	Free (open-source)
VS Code IDE	Free (open-source)
Razorpay (test mode for development)	Free; transaction fees in production (~2%)
xhtml2pdf library	Free (open-source)
Development hardware (laptop/PC)	Already available
Internet connection	Already available

Web hosting for production (optional)	₹200–₹500/month (shared hosting)
Total Development Cost	**Negligible (near zero)**

Benefits: The system eliminates double bookings (reducing guest complaints and refunds), saves staff time through automation, enables online bookings (increasing revenue), provides instant reports (improving decision-making), and offers 24/7 chatbot support (enhancing guest satisfaction).

Conclusion: The project is economically feasible. It uses free, open-source tools and provides significant operational benefits relative to its negligible development cost.

7. Tools / Platform, Hardware and Software Requirements

7.1 Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS
Programming Language	Python 3.10+ (3.x series)
Web Framework	Django 6.0
Database	PostgreSQL 16 (open-source, advanced relational database)
Frontend – Markup & Styling	HTML5, CSS3, Bootstrap 5.3
Frontend – Scripting	JavaScript (ES6), AJAX (Fetch API)
Payment Gateway	Razorpay Python SDK (`razorpay` package)

PDF Generation	xhtml2pdf 0.2+ (Python library)
Email Backend	Django SMTP Email Backend (Gmail SMTP)
Environment Configuration	python-decouple (for managing secrets via `.env` file)
IDE / Code Editor	Visual Studio Code (VS Code)
Web Browser (Testing)	Google Chrome 120+, Microsoft Edge 120+, Mozilla Firefox 120+
Version Control	Git 2.x
Virtual Environment	Python `venv` module

7.2 Python Package Dependencies

The following Python packages are installed in the virtual environment (as specified in `requirements.txt`):

Package	Purpose
`django`	Web framework
`razorpay`	Razorpay payment gateway SDK
`xhtml2pdf`	HTML to PDF conversion
`reportlab`	PDF rendering engine (dependency of xhtml2pdf)
`python-decouple`	Environment variable management

`Pillow`	Image processing for ImageField
`pdfplumber`	PDF text extraction utility

7.3 Hardware Requirements

Component	Minimum Specification	Recommended
Processor	Intel Core i3 / Dual-core equivalent	Intel Core i5 or above
RAM	4 GB	8 GB or above
Hard Disk Space	20 GB free	50 GB SSD
Display Resolution	1366 × 768	1920 × 1080 (Full HD)
Internet Connection	Required (for payment gateway, email, and deployment)	Broadband 10 Mbps+
Input Devices	Keyboard and Mouse	Standard peripherals

8. Analysis Document

8.1 Software Requirement Specification (SRS)

8.1.1 Purpose

The purpose of this Software Requirement Specification is to describe the functional and non-functional requirements of the Hotel Room Booking Website with Assistant Chatbot. This document serves as a reference for the development team and the project guide, and defines the expected behaviour of the system.

8.1.2 Scope

The system is a web-based application for hotel room booking and management. It serves two categories of users: **Administrators** (hotel staff) and **Customers** (guests). The application will be accessible via a web browser and will communicate with a PostgreSQL database on the server.

8.1.3 Functional Requirements – Administrator

Req. ID	Requirement	Description
FR-A01	Admin Login	The admin shall be able to log in using

		Django's built-in authentication system (username and password).
FR-A02	Admin Logout	The admin shall be able to securely log out, destroying the session.
FR-A03	Add Room Type	The admin shall be able to add a new room type with name, description, and image.
FR-A04	View Room Types	The admin shall be able to view all room types in a tabular format with edit and delete actions.
FR-A05	Edit Room Type	The admin shall be able to modify the name, description, and image of an existing room type.
FR-A06	Delete Room Type	The admin shall be able to delete a room type (subject to referential integrity).
FR-A07	Add Room	The admin shall be able to add a new room linked to a room type, with name, price, description, and image.
FR-A08	View Rooms	The admin shall be able to view all rooms

		with their type, price, and actions.
FR-A09	Edit Room	The admin shall be able to modify room details.
FR-A10	Delete Room	The admin shall be able to delete a room (subject to referential integrity).
FR-A11	Add Staff	The admin shall be able to add staff with name, designation, social links, and photo.
FR-A12	View Staff	The admin shall view all staff with details and actions.
FR-A13	Edit Staff	The admin shall be able to modify staff details.
FR-A14	Delete Staff	The admin shall be able to delete a staff record.
FR-A15	View Contact Messages	The admin shall view all customer contact messages.
FR-A16	Delete Contact Messages	The admin shall be able to delete processed contact messages.
FR-A17	View Bookings	The admin shall view all booking records

		with customer, room, dates, and price.
FR-A18	View Payment Totals	The admin shall view all payment records with customer, mobile, and total amount.
FR-A19	Manage Chatbot	The admin shall manage chatbot responses, conversations, and notifications via Django Admin.

8.1.4 Functional Requirements – Customer

Req. ID	Requirement	Description
FR-C01	User Registration	A new customer shall be able to register with username, email, password, and confirm password.
FR-C02	User Sign-In	A registered customer shall be able to sign in with username and password.
FR-C03	User Sign-Out	A signed-in customer shall be able to sign out, destroying the session.
FR-C04	Browse Room Categories	The customer shall see room type cards on the home page and navigate to rooms

		of a specific type.
FR-C05	View Room Details	The customer shall see room name, image, price, description, and a "Book" button for each room.
FR-C06	Advanced Search	The customer shall search rooms by name, type, min price, max price, check-in date, and check-out date using an AJAX-powered search panel.
FR-C07	Book a Room	The customer shall fill in booking details (name, email, dates, adults, children, special request) and submit a booking.
FR-C08	Overlap Detection	The system shall check for overlapping bookings and reject the booking if the room is already booked for the requested dates.
FR-C09	View Booking Cart	The customer shall see booked rooms with subtotal, tax, and total amount.
FR-C10	Delete Booking	The customer shall be able to remove a booking from the cart before payment.

FR-C11	Make Payment	The customer shall pay the total amount via Razorpay (credit/debit card, UPI, net banking).
FR-C12	Submit Contact Enquiry	The customer shall submit a contact form with name, email, phone, subject, and message.
FR-C13	Chatbot Interaction	The customer shall send text messages to the chatbot and receive contextual responses.
FR-C14	Rate Chatbot Response	The customer shall rate chatbot responses on a 1–5 scale.
FR-C15	Password Reset via OTP	The customer shall reset their password using an OTP sent to their registered email.
FR-C16	Receive Email Confirmation	The customer shall receive a booking confirmation email after successful booking.

8.1.5 Non-Functional Requirements

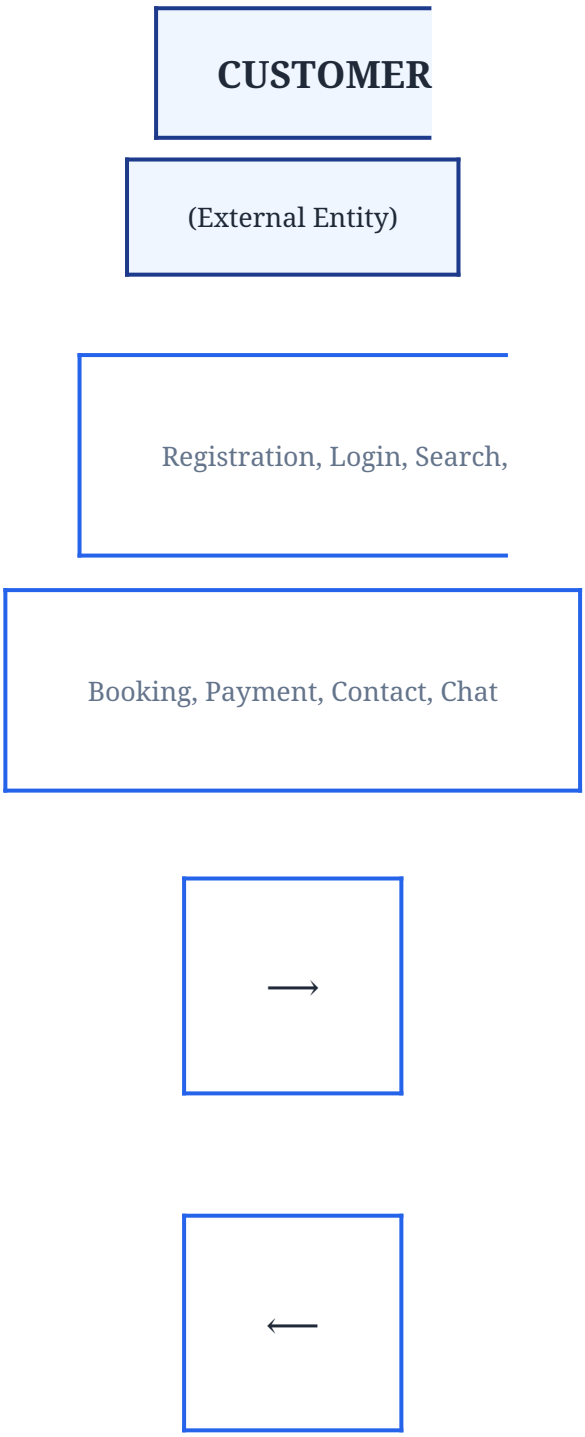
Category	Requirement
Performance	Web pages shall load within 2–3 seconds on a broadband connection. AJAX search results shall appear without a full page reload.
Security	CSRF protection on all forms, ORM-based parameterised queries to prevent SQL injection, template auto-escaping to prevent XSS, session-based authentication, password masking on input fields.
Usability	Responsive design that works on desktop (1920px), tablet (768px), and mobile (375px) screen widths. Consistent navigation and layout across all pages.
Reliability	Input validation on all forms (required fields, numeric checks, date format checks). Graceful error handling with user-friendly messages. Date-overlap detection prevents booking conflicts.
Maintainability	Modular Django app structure (Backend app, webapp). Separation of concerns (models, views,

	templates, URLs). Code documentation with comments.
Scalability	Database uses PostgreSQL for production-grade scalability, concurrent write support, and cloud deployment readiness. New modules can be added as new Django apps. API endpoints support future mobile app development.
Portability	The application runs on Windows, Linux, and macOS. It requires only Python, Django, and a web browser.

8.2 Data Flow Diagrams (DFD)

8.2.1 DFD Level-0 (Context Diagram)

The context diagram shows the system as a single process with two external entities: **Customer** and **Administrator**. It represents the highest-level view of data flow in and out of the system.


CUSTOMER
(External Entity)
Registration, Login, Search,
Booking, Payment, Contact, Chat
⟶
⟵

Room details, Booking confirmation,

Search results, Chat responses, PDF

Hotel Room Booking System

(Process 0.0)

Room/Staff/Type CRUD,

View bookings, View payments

←

Figure 8.1 – DFD Level-0 (Context Diagram)

8.2.2 DFD Level-1

The Level-1 DFD decomposes the single process of the context diagram into eight sub-processes corresponding to the eight functional modules of the system. It also shows the data stores (database tables) that each process reads from and writes to.

Process	Input Data Flow	Output Data Flow	Data Store(s)

	Username,	Session token,	
1.0 User Registration & Login	Email, Password, Login credentials	Access granted/denied, Success/Error message	D1: USER_MASTER (Registerdb)
2.0 Room Type Management	Room type name, image, description	Updated room type catalogue, Success message	D2: ROOMTYPE_MASTER (roomtypedb)
3.0 Room Management	Room name, type FK, image, price, description	Updated room inventory, Success message	D3: ROOM_MASTER (roomnamedb), D2
4.0 Staff Management	Staff name, designation, image, social links	Staff list for website & admin	D4: STAFF_MASTER (staffdb)
5.0 Room Search &	Search filters, dates,	Search results (JSON), Booking	D3, D5: BOOKING_MASTER

Booking	guest details Booking total,	confirmation/rejection, Email Razorpay	(bookingdb), D1
6.0 Payment Processing	customer mobile, payment method	order, Payment receipt, Success message	D6: PAYMENT_TOTAL (Totaldb), D5
7.0 Contact Management	Name, email, phone, subject, message	Stored enquiry, Success message	D7: CUSTOMER_CONTACT (customercontactdb)
8.0 Assistant Chatbot	User text message, Username	Classified response, Room cards, Suggestions	D8: ChatbotConversation, D9: ChatbotResponse, D3, D4, D5

Table 8.1 – DFD Level-1 Process Summary

8.3 Entity-Relationship (ER) Diagram

The ER diagram identifies the entities, their attributes, primary keys, and the relationships (with cardinality) between entities in the system.

8.3.1 Entities and Their Key Attributes

Entity	Primary Key	Key Attributes
roomtypedb	id (Auto PK)	ROOMTYPE, ROOMTYPEIMAGE, DESCRIPTION
roomnamedb	id (Auto PK)	ROOMTYPE (FK), ROOMNAME, ROOMIMAGE, ROOMPRICE, ROOMDESCRIPTION
Registerdb	id (Auto PK)	USERNAME, PASSWORD, CONFIRMPASSWORD, EMAIL
bookingdb	id (Auto PK)	CUSTOMER (FK), CUSTOMERNAME, CONTACTEMAIL, CHECKIN, CHECKOUT, TOTALADULTS, TOTALCHILDS, SELECTROOM (FK),

		SPECIALREQUEST, TOTALPRICE
Totaldb	id (Auto PK)	BOOKING (FK), CUSTOMERNAME, MOBILE, TOTALPRICE
staffdb	id (Auto PK)	STAFFNAME, STAFFDESIGNATION, STAFFACEBOOK, STAFFINSTA, STAFFIMAGE
customercontactdb	id (Auto PK)	CONTACTNAME, CONTACTNUMBER, CONTACTEMAIL, CONTACTSUBJECT, CONTACTMESSAGE
ChatbotConversation	id (Auto PK)	username, user_message, bot_response, query_type, rating, created_at
ChatbotResponse	id (Auto PK)	Predefined response templates
Notification	id (Auto PK)	username, notification_type, title, message, is_read, related_object_id, created_at

8.3.2 Relationships and Cardinality

Relationship	Cardinality	Description
roomtypedb ↔️ roomnamedb	1 : N (One-to-Many)	One room type can have many rooms. Each room belongs to exactly one room type.
roomnamedb ↔️ bookingdb	1 : N (One-to-Many)	One room can have many bookings (over time). Each booking is for exactly one room.
Registerdb ↔️ bookingdb	1 : N (One-to-Many)	One registered user can make many bookings. Each booking is optionally linked to one user.
bookingdb ↔️ Totaldb	1 : N (One-to-Many)	One booking can have payment records. Each payment is linked to one booking.
staffdb	Independent	No direct relationship with

		other entities.
customercontactdb	Independent	No direct relationship with other entities.
ChatbotConversation	Independent	Stores username as text field; no FK to Registerdb.

8.4 Data Dictionary

The data dictionary provides a detailed description of every field in every table of the database, including field name, data type, constraints, and purpose.

Table 1: ROOMTYPE_MASTER (roomtypedb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each room type
ROOMTYPE	VARCHAR	100	NULL allowed	Name of the room type (e.g., Deluxe,

				Suite, Standard)
ROOMTYPEIMAGE	VARCHAR (ImageField)	255	NULL allowed, upload_to="roomtypeimages"	File path to the room type image
DESCRIPTION	VARCHAR	100	NULL allowed	Brief description of the room type

Table 2: ROOM_MASTER (roomnamedb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique room identifier
ROOMTYPE	INTEGER	—	FOREIGN KEY → roomtypedb(id), CASCADE,	Reference to the room's type category

			NULL allowed	
ROOMNAME	VARCHAR	100	NULL allowed	Name or number of the room
ROOMIMAGE	VARCHAR (ImageField)	255	NULL allowed, upload_to="roomimage"	File path to the room's image
ROOMPRICE	INTEGER	—	NULL allowed	Price per night in INR
ROOMDESCRIPTION	VARCHAR	100	NULL allowed	Description of the room's features and amenities

Table 3: USER_MASTER (Registerdb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO	Unique user identifier

			INCREMENT	
USERNAME	VARCHAR	100	NULL allowed	User's chosen display name
PASSWORD	VARCHAR	100	NULL allowed	User's login password
CONFIRMPASSWORD	VARCHAR	100	NULL allowed	Password confirmation (for validation)
EMAIL	VARCHAR (EmailField)	100	NULL allowed	User's email address

Table 4: BOOKING_MASTER (bookingdb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique booking identifier
CUSTOMER	INTEGER	—	FOREIGN KEY →	Link to the

			Registerdb(id), SET_NULL, NULL allowed	registered customer who made the booking
CUSTOMERNAME	VARCHAR	100	NULL allowed	Customer's name (for display purposes)
CONTACTEMAIL	VARCHAR (EmailField)	100	NULL allowed	Email for booking communication
CHECKIN	DATE	—	NULL allowed	Check-in date
CHECKOUT	DATE	—	NULL allowed	Check-out date
TOTALADULTS	INTEGER	—	NULL allowed	Number of adult guests
TOTALCHILDS	INTEGER	—	NULL allowed	Number of child guests
SELECTROOM	INTEGER	—	FOREIGN KEY → roomnamedb(id)	The specific room being

			, CASCADE, NULL allowed	booked
SPECIALREQUEST	TEXT	—	NULL allowed	Any special requests from the guest
TOTALPRICE	INTEGER	—	NULL allowed	Total booking price in INR

Table 5: PAYMENT_TOTAL (Totaldb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique payment identifier
BOOKING	INTEGER	—	FOREIGN KEY → bookingdb(id), CASCADE, NULL allowed	Link to the associated booking
CUSTOMERNAME	VARCHAR	100	NULL allowed	Customer's name

MOBILE	VARCHAR	15	NULL allowed	Customer's mobile number
TOTALPRICE	INTEGER	—	NULL allowed	Total amount paid in INR

Table 6: CUSTOMER_CONTACT (customercontactdb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique contact identity
CONTACTNAME	VARCHAR	100	NULL allowed	Customer's name
CONTACTNUMBER	VARCHAR	15	NULL allowed	Customer's phone number
CONTACTEMAIL	VARCHAR (EmailField)	100	NULL allowed	Customer's email address

CONTACTSUBJECT	VARCHAR	200	NULL allowed	Subject of the enquiry
CONTACTMESSAGE	TEXT	—	NULL allowed	Full message body

Table 7: STAFF_MASTER (staffdb)

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique staff identifier
STAFFNAME	VARCHAR	100	NULL allowed	Staff member's name
STAFFDESIGNATION	VARCHAR	100	NULL allowed	Job title / role
STAFFACEBOOK	VARCHAR	100	NULL allowed	Facebook profile URL
STAFFINSTA	VARCHAR	100	NULL allowed	Instagram profile URL

			NULL	
STAFFIMAGE	VARCHAR (ImageField)	255	allowed, upload_to="staffimage"	Staff photo file path

Table 8: ChatbotConversation

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique conversation identifier
username	VARCHAR	100	NOT NULL	User who initiated the conversation
user_message	TEXT	—	NOT NULL	The user's query message
bot_response	TEXT	—	NOT NULL	The chatbot's response
query_type	VARCHAR	20	NOT NULL	Classification: room, staff, booking, or

				general
rating	INTEGER	—	NULL allowed, range 1–5	User's satisfaction rating
created_at	DATETIME	—	AUTO (auto_now_add=True)	Timestamp of conversation

Table 9: ChatbotResponse

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique response identifier
keywords	TEXT	—	NOT NULL	Comma-separated keywords that trigger this response
response_text	TEXT	—	NOT NULL	The response to display
category	VARCHAR	20	NOT NULL	Response category

				(room, booking, staff, general)
priority	INTEGER	—	DEFAULT 0	Priority for response selection
is_active	BOOLEAN	—	DEFAULT True	Whether this response is active

Table 10: Notification

Field Name	Data Type	Size	Constraints	Description
id	INTEGER	—	PRIMARY KEY, AUTO INCREMENT	Unique notification identifier
username	VARCHAR	100	NOT NULL	Target user for the notification
notification_type	VARCHAR	20	NOT NULL	Type: booking, cancellation, alert, info
title	VARCHAR	200	NOT NULL	Notification headline

message	TEXT	—	NOT NULL	Full notification message
is_read	BOOLEAN	—	DEFAULT False	Whether the user has read it
related_object_id	INTEGER	—	NULL allowed	ID of related room/booking
created_at	DATETIME	—	AUTO (auto_now_add=True)	Creation timestamp

9. Design Document

9.1 Modularization Details – 8 Modules

The system is divided into eight functional modules. Each module handles a specific set of related features. The modules are described below with their purpose, functions, input/output, and pseudocode.

Module 1 – Secure Login & User Management

Purpose: This module allows only authorized users to access customer features or the admin backend. It verifies the username and password against the registered users table and stores the username in the session so that unauthorized persons cannot view, modify, or delete data.

Functions Covered:

- User registration (sign-up) with username, email, password, and confirm password
- User sign-in with session creation
- User sign-out with session destruction
- Admin login using Django's built-in `authenticate()` and `login()` functions
- Admin logout using Django's `logout()` function

- Password reset using email OTP verification
- OTP generation, email dispatch, OTP verification, and password update

Pseudocode – User Registration:

```
FUNCTION save_user(request):

    IF request method is POST:

        GET username, email, password, confirm_password from form

        CREATE new Registerdb object with these values

        SAVE object to database

        DISPLAY success message "Signup successful"

        REDIRECT to sign-in page

    END IF

END FUNCTION
```

Pseudocode – User Login:

```
FUNCTION user_login(request):

    IF request method is POST:

        GET username, password from form

        IF Registerdb record exists WHERE USERNAME=username AND
PASSWORD=password:

            SET session['USERNAME'] = username
```

```
            SET session['PASSWORD'] = password

            DISPLAY success message "Signin successful"

            REDIRECT to home page

        ELSE:

            DISPLAY warning "Signin failed"

            REDIRECT to sign-in page

        END IF

    END IF

END FUNCTION
```

Pseudocode – Password Reset via OTP:

```
FUNCTION reset_password_email_verification(request):

    IF request method is POST:

        GET email from form

        IF Registerdb record exists WHERE EMAIL=email:

            GENERATE random OTP (5 digits)

            STORE OTP in session

            SEND email to user with OTP

            DISPLAY OTP input form
```

```
        ELSE:
            DISPLAY error "Invalid email"
        END IF
    END IF
END FUNCTION


FUNCTION verify_otp(request, user_id):
    GET user_otp from form
    GET session_otp from session
    IF user_otp == session_otp:
        DISPLAY new password form
    ELSE:
        DISPLAY error "OTP does not match"
    END IF
END FUNCTION


FUNCTION reset_password(request, username):
    GET password1, password2 from form
    IF password1 == password2:
```

```
        UPDATE Registerdb SET PASSWORD = password1 WHERE USERNAME =
username

        SEND email notification about password change

        DISPLAY success message

        REDIRECT to sign-in page

    ELSE:

        DISPLAY error "Passwords do not match"

    END IF

END FUNCTION
```

Module 2 – Room Type & Room Management

Purpose: This module enables the admin to insert, update, and delete room types (e.g., Deluxe, Suite, Standard) and individual rooms including images, descriptions, and prices, so that the online catalogue always matches the actual inventory of the hotel. Rooms are linked to their room type via a foreign key relationship.

Functions Covered:

- Add room type with name, description, and image upload
- Display all room types in a table with edit/delete buttons
- Edit room type details and image
- Delete room type (cascade deletes associated rooms)
- Add room with name, type (dropdown FK), price,

description, and image

- Display all rooms with type, price, and actions
- Edit room details
- Delete room

Pseudocode – Add Room Type:

```
FUNCTION addRoomType(request):

    IF request method is POST:

        GET roomtype_name, description, image from form

        CREATE new roomtypedb object

        SET object fields

        SAVE to database

        DISPLAY success message

        REDIRECT to display room types page

    END IF

END FUNCTION
```

Pseudocode – Add Room (with FK):

```
FUNCTION addRoom(request):

    IF request method is POST:

        GET room_name, room_type_id, price, description, image from
form
```

```
        GET room_type_object =
roomtypedb.objects.get(id=room_type_id)

        CREATE new roomnamedb object

        SET ROOMTYPE = room_type_object  (Foreign Key assignment)

        SET ROOMNAME, ROOMPRICE, ROOMDESCRIPTION, ROOMIMAGE

        SAVE to database

        DISPLAY success message

    END IF

END FUNCTION
```

Module 3 – Staff Management

Purpose: When new staff members join or existing staff details change, this module is used. It allows the admin to maintain staff name, designation, social-media links (Facebook, Instagram), and photos. These details are displayed on the About and Our Team pages so that customers can know about the team behind the hotel services.

Functions Covered: Add staff, display all staff, edit staff, delete staff.

Pseudocode – Add Staff:

```
FUNCTION addStaff(request):

    IF request method is POST:
```

```
        GET staff_name, designation, facebook, instagram, image
from form

        CREATE new staffdb object with these values

        SAVE to database

        DISPLAY success message

        REDIRECT to display staff page

    END IF

END FUNCTION
```

Module 4 – Customer Contact Management

Purpose: When a visitor sends a message through the contact form, this module stores the details—customer name, email, mobile number, subject, and message—so that the admin can read, respond to, and delete messages from a single backend page, improving hotel–guest communication.

Pseudocode – Save Contact:

```
FUNCTION save_customer_contact(request):

    IF request method is POST:

        GET name, email, number, subject, message from form

        CREATE new customercontactdb object

        SAVE to database
```

```
        DISPLAY success message "Message sent successfully"
        REDIRECT to contact page
    END IF
END FUNCTION
```

Module 5 – Room Search & Booking Management

Purpose: This is one of the main functional modules. It allows a logged-in customer to search rooms by room type, room name, date range, and price and then place a booking request. The module checks availability by validating date-range overlaps for the selected room, prevents double booking, stores booking details such as check-in, check-out, adults, children, special requests, and calculates the total price. An advanced search interface using AJAX provides real-time filtering without full page reload.

Functions Covered:

- Browse rooms by category (filter by room type)
- Advanced search with AJAX (room name, type, min price, max price, check-in, check-out)
- Book a room with overlap detection
- View booking cart with subtotal, tax, and total calculation
- Delete a booking from the cart

Pseudocode – Save Booking with Overlap Detection:

```
FUNCTION save_booking(request):
```

```
    IF request method is POST:

        GET customer_name, email, checkin, checkout, adults,
children,

            room_id, special_request, total_price from form



        TRY:

            room_object = roomnamedb.objects.get(id=room_id)

        EXCEPT DoesNotExist:

            DISPLAY error "Invalid room selection"

            REDIRECT to save room page

            RETURN



        // Check for overlapping bookings

        overlapping = bookingdb.objects.filter(

            SELECTROOM = room_object,

            CHECKIN <= checkout,

            CHECKOUT >= checkin

        )
```

```
        IF overlapping exists:

            DISPLAY error "Room already booked for these dates"

            REDIRECT to save room page

        ELSE:

            // Get customer FK if logged in

            customer_obj = NULL

            IF session has 'USERNAME':

                TRY:

                    customer_obj =
Registerdb.objects.get(USERNAME=session['USERNAME'])

                EXCEPT: pass



            CREATE new bookingdb object with all fields

            SET CUSTOMER = customer_obj  (FK)

            SET SELECTROOM = room_object  (FK)

            SAVE to database



            DISPLAY success "Room booked successfully"

            SEND confirmation email to customer
```

```
            REDIRECT to save room page

        END IF

    END IF

END FUNCTION
```

Pseudocode – Cart Calculation:

```
FUNCTION save_room_page(request):

    GET all bookings WHERE CUSTOMERNAME = session['USERNAME']

    SET subtotal = 0, tax = 0, total = 0



    FOR each booking in bookings:

        subtotal = subtotal + booking.TOTALPRICE

    END FOR



    IF subtotal > 5000:

        tax = 10  (percentage)

    ELSE:

        tax = 20  (flat)
```

```
    total = subtotal + tax

    RENDER template with bookings, subtotal, tax, total

END FUNCTION
```

Module 6 – Payment & Total Amount Module

Purpose: After confirming the booking, this module calculates the final total amount to be paid by the customer. It integrates with the Razorpay payment gateway to create an order, process online payment, and stores the final paid amount, customer name, and mobile in the PAYMENT_TOTAL table for future reference and financial reporting.

Pseudocode – Payment Processing:

```
FUNCTION payment_page(request):

    GET latest Totaldb record (most recent payment entry)

    SET amount = record.TOTALPRICE * 100  (Razorpay uses paise)



    IF request method is POST:

        SET currency = 'INR'

        CREATE Razorpay client with API key and secret

        CREATE order = client.order.create(amount, currency,
capture='1')
```

```
    END IF



    RENDER payment template with customer details and amount

END FUNCTION



FUNCTION save_payment_input(request):

    IF request method is POST:

        GET customer_name, mobile, total_price from form

        GET last_booking for this customer

        CREATE new Totaldb object

        SET BOOKING = last_booking  (FK)

        SET CUSTOMERNAME, MOBILE, TOTALPRICE

        SAVE to database

        REDIRECT to payment page

    END IF

END FUNCTION
```

Module 7 – Notification, Email & PDF Report Module

Purpose: This module handles user-friendly communication and

reporting features. It sends booking confirmation emails, password reset OTP emails, and password-change alerts to customers. It generates PDF documents for booking details using xhtml2pdf. It also uses JavaScript-based instant notifications to inform users about successful operations like booking, message submission, or data updates.

Functions Covered:

- Send booking confirmation email via Django SMTP
- Send OTP email for password reset
- Send password-changed notification email
- Generate PDF report from HTML template using xhtml2pdf
- Display Django `messages` framework notifications (success, warning, error)
- Get notifications API endpoint (JSON)
- Mark notification as read API endpoint

Pseudocode – PDF Generation:

```
FUNCTION generate_pdf(request):

    GET username from session (default 'Guest')

    SET context = { username, booking_data }

    RENDER HTML template to string using render_to_string()

    CREATE HTTP response with content_type = 'application/pdf'

    SET Content-Disposition header for download

    CALL pisa.CreatePDF(html_string, dest=response)
```

```
    IF errors:
        RETURN error response
    ELSE:
        RETURN PDF response
    END IF
END FUNCTION
```

Module 8 – Assistant Chatbot Module

Purpose: The assistant chatbot provides an interactive way for customers to communicate with the hotel website. It can answer questions about available rooms, prices, facilities, staff information, booking policies, check-in/out times, and guide the user to the correct page or action, thereby reducing manual support work for the staff and improving customer satisfaction. Conversations are stored for analytics and users can rate responses.

Functions Covered:

- `chatbot_query()` – Main API endpoint (POST) that receives user message and returns response
- `generate_chatbot_response()` – Core engine that classifies query and generates response
- `handle_room_query()` – Handles room-related questions with database lookup
- `handle_staff_query()` – Handles staff-related questions

- `handle_booking_query()` – Handles booking-related questions
- `handle_general_query()` – Handles greetings, help, and general questions
- `rate_response()` – API endpoint (POST) for user to rate a response

Pseudocode – Chatbot Query Classification:

```
FUNCTION generate_chatbot_response(user_message, username):

    DEFINE room_keywords = ['room', 'price', 'availability',
'luxury', 'deluxe', 'suite']

    DEFINE staff_keywords = ['staff', 'team', 'manager',
'receptionist']

    DEFINE booking_keywords = ['booking', 'reserve', 'book',
'check-in', 'check-out']



    CONVERT user_message to lowercase



    IF any room_keyword IN user_message:

        RETURN handle_room_query(user_message, username)

    ELSE IF any staff_keyword IN user_message:

        RETURN handle_staff_query(user_message, username)

    ELSE IF any booking_keyword IN user_message:

        RETURN handle_booking_query(user_message, username)
```

```
    ELSE:
        RETURN handle_general_query(user_message, username)
    END IF
END FUNCTION


FUNCTION handle_room_query(user_message, username):
    IF 'price' IN user_message:
        response = "We offer rooms at various price points..."
        suggestions = ['Show budget rooms', 'Show luxury rooms']
    ELSE IF 'luxury' IN user_message:
        QUERY rooms =
roomnamedb.objects.filter(ROOMTYPE__ROOMTYPE__icontains='luxury')
        BUILD room_list with id, name, type, price, image for each
room
        response = "Here are our luxury rooms."
        data = room_list
    ELSE:
        COUNT total rooms
        response = "We have {count} rooms available..."
    END IF
```

```
    RETURN {response, query_type='room', suggestions, data}

END FUNCTION
```

9.2 Database Design and Normalization

9.2.1 Normalization

The database design follows normalization principles to reduce data redundancy and ensure data integrity.

First Normal Form (1NF): All tables satisfy 1NF because:

- Each table has a primary key (auto-generated `id` field).
- All columns contain atomic (indivisible) values—there are no repeating groups or multi-valued attributes.
- Each row is unique, identified by its primary key.

Second Normal Form (2NF): All tables satisfy 2NF because:

- They are already in 1NF.
- All non-key attributes are fully functionally dependent on the primary key. Since all primary keys are single-column auto-incremented IDs, there are no partial dependencies.

Third Normal Form (3NF): The database is in 3NF because:

- It is in 2NF.
- There are no transitive dependencies. For example, in the `bookingdb` table, the `SELECTROOM` field is a foreign key to `roomnamedb`—the room's name, type, and price are not duplicated in the

booking table. Similarly, `CUSTOMER` is a foreign key to `Registerdb` —the customer's email and password are not stored in the booking table.

- The `CUSTOMERNAME` field in `bookingdb` is a denormalized convenience field for display purposes, which is an acceptable trade-off for query performance.

9.2.2 Referential Integrity

Django's ORM enforces referential integrity through the `on_delete` parameter on ForeignKey fields:

Foreign Key	Parent Table	on_delete Action	Effect
roomnamedb.ROOMTYPE	roomtypedb	CASCADE	Deleting a room type deletes all its rooms
bookingdb.CUSTOMER	Registerdb	SET_NULL	Deleting a user sets the booking's customer to NULL (booking

			preserved)
bookingdb.SELECTROOM	roomnamedb	CASCADE	Deleting a room deletes all its bookings
Totaldb.BOOKING	bookingdb	CASCADE	Deleting a booking deletes its payment records

9.3 Procedural Design / Process Logic

9.3.1 Booking Workflow

1. Customer signs in using username and password → session is created.
2. Customer browses room categories on the home page → clicks on a room type.
3. System displays all rooms of that type with images, prices, and "Book" buttons.
4. Customer clicks "Book" on a specific room → booking form is

displayed.

5. Customer fills in: name, email, check-in date, check-out date, adults, children, special request.
6. System calculates total price (room price × number of nights).
7. Customer submits the booking form.
8. System checks for overlapping bookings: `CHECKIN ≤ checkout AND CHECKOUT ≥ checkin`.
9. If overlap detected → display error "Room already booked for these dates" → redirect.
10. If no overlap → save booking → send confirmation email → redirect to cart page.
11. Cart page shows all bookings with subtotal, tax, and total.
12. Customer enters mobile number and proceeds to payment page.
13. Razorpay order is created → payment popup appears → customer pays.
14. Payment record is saved in Totaldb with booking FK.

9.3.2 Advanced Search Workflow

1. User opens the advanced search page.
2. User enters search criteria: room name (text), room type (dropdown), minimum price, maximum price, check-in date, check-out date.
3. JavaScript captures the form inputs and constructs a query string.
4. JavaScript sends an AJAX GET request to `/Backend/api/search-rooms/?params`.

5. Django view (`search_rooms`) receives the request and builds a queryset filter chain.
6. If check-in and check-out dates are provided, the view excludes rooms that have overlapping bookings.
7. The filtered rooms are serialized to JSON (id, name, type, price, image URL, description).
8. JavaScript receives the JSON response and dynamically renders room cards in the results area without reloading the page.

9.3.3 Chatbot Workflow

1. User clicks the floating chatbot icon on any page → chat window opens.
2. User types a message and clicks "Send" or presses Enter.
3. JavaScript sends an AJAX POST request to `/Backend/api/chatbot/` with the message and username.
4. Django view parses the JSON body, extracts the message.
5. The `generate_chatbot_response()` function classifies the query using keyword matching.
6. Based on the classification (room/staff/booking/general), the appropriate handler function is called.
7. The handler queries the database if needed (e.g., fetching rooms, staff) and builds a response object.
8. The conversation is saved in ChatbotConversation.
9. A JSON response with text, suggestions, and optional data (room cards) is returned.
10. JavaScript renders the bot's response in the chat window, including suggestion chips and room cards.

11. User can rate the response using a star rating, which is sent via AJAX to `/Backend/api/rate-response/`.

9.3.4 Password Reset Workflow

1. User clicks "Forgot Password" on the sign-in page.
2. Email verification page is displayed → user enters registered email.
3. System verifies the email exists in Registerdb.
4. If valid → generate a random 5-digit OTP → store in session → send OTP email.
5. OTP input page is displayed → user enters the OTP received via email.
6. System compares the entered OTP with the session OTP.
7. If match → new password form is displayed.
8. User enters new password and confirms it.
9. If passwords match → update Registerdb → send password-changed notification email → redirect to sign-in.
10. If passwords don't match → display error → allow retry.

9.4 User Interface Design

The user interface is designed with a focus on simplicity, responsiveness, and visual consistency. The design principles followed are:

- **Responsive Layout:** Bootstrap 5.3's grid system and utility classes ensure that all pages adapt seamlessly to desktop (≥1200px), tablet (768px–1199px), and mobile (<768px)

screen sizes.

- **Consistent Navigation:** A fixed-top navigation bar with the hotel logo, menu items (Home, About, Services, Rooms, Contact), and user authentication links (Sign In, Sign Out) is present on all customer-facing pages.
- **Card-Based Design:** Room types, rooms, and staff profiles are displayed as Bootstrap cards with images, descriptions, and action buttons.
- **Form Design:** All input forms use Bootstrap form components with labels, placeholders, and validation feedback.
- **Color Scheme:** A professional hotel-themed colour palette with primary blues, neutral greys, and accent colours for buttons and highlights.
- **Admin Backend:** The admin backend uses a separate template set with a sidebar navigation, tabular data displays, and modal confirmations for delete actions.

Customer-Facing Pages

1. Home page – Room type cards, hero section, chatbot widget
2. About page – Hotel description, staff profiles grid
3. Services page – Hotel services and amenities
4. Rooms page – All room categories with images
5. Filtered rooms page – Rooms of a specific type with "Book" buttons
6. Advanced search page – Filter panel + AJAX results grid
7. Booking form page – Input form for booking details
8. Booking cart page – Booked items with subtotal, tax, total

9. Payment page – Razorpay integration with order summary
10. Contact page – Contact form with name, email, phone, subject, message
11. Sign-up page – Registration form
12. Sign-in page – Login form
13. Password reset pages – Email verification → OTP → New password
14. Our Team page – Staff profiles with social links
15. Testimonials page – Customer reviews

Admin Backend Pages

1. Admin login page
2. Dashboard / index page
3. Add room type page (form)
4. Display room types page (table)
5. Edit room type page (form)
6. Add room page (form with type dropdown)
7. Display rooms page (table)
8. Edit room page (form)
9. Add staff page (form)
10. Display staff page (table)
11. Edit staff page (form)
12. Customer contact messages page (table)
13. Booked details page (table)
14. Payment totals page (table)
15. Django Admin – Chatbot responses, conversations,

notifications

10. Source Code (Key Modules)

This section presents the source code of the most important modules of the project. The complete source code is available in the project files submitted along with this report.

10.1 Django Models – webapp/models.py

```
from django.db import models

from Backend.models import roomnamedb



class customercontactdb(models.Model):

    CONTACTNAME = models.CharField(max_length=100, null=True,
blank=True)

    CONTACTNUMBER = models.CharField(max_length=15, null=True,
blank=True)

    CONTACTEMAIL = models.EmailField(max_length=100, null=True,
blank=True)

    CONTACTSUBJECT = models.CharField(max_length=200, null=True,
blank=True)

    CONTACTMESSAGE = models.TextField(null=True, blank=True)
```

```
    def __str__(self):
        return self.CONTACTNAME or "No Name"


class Registerdb(models.Model):
    USERNAME = models.CharField(max_length=100, null=True,
blank=True)
    PASSWORD = models.CharField(max_length=100, null=True,
blank=True)
    CONFIRMPASSWORD = models.CharField(max_length=100, null=True,
blank=True)
    EMAIL = models.EmailField(max_length=100, null=True,
blank=True)


    def __str__(self):
        return self.USERNAME or "No Username"


class bookingdb(models.Model):
    CUSTOMER = models.ForeignKey(
        Registerdb, on_delete=models.SET_NULL,
        null=True, blank=True, related_name='bookings'
```

```
    )

    CUSTOMERNAME = models.CharField(max_length=100, null=True,
blank=True)

    CONTACTEMAIL = models.EmailField(max_length=100, null=True,
blank=True)

    CHECKIN = models.DateField(null=True, blank=True)

    CHECKOUT = models.DateField(null=True, blank=True)

    TOTALADULTS = models.IntegerField(null=True, blank=True)

    TOTALCHILDS = models.IntegerField(null=True, blank=True)

    SELECTROOM = models.ForeignKey(

        roomnamedb, on_delete=models.CASCADE,

        null=True, blank=True, related_name='bookings'

    )

    SPECIALREQUEST = models.TextField(null=True, blank=True)

    TOTALPRICE = models.IntegerField(null=True, blank=True)



    def __str__(self):

        return f"{self.CUSTOMERNAME} - {self.SELECTROOM}"



class Totaldb(models.Model):
```

```
    BOOKING = models.ForeignKey(

        bookingdb, on_delete=models.CASCADE,

        null=True, blank=True, related_name='totals'

    )

    CUSTOMERNAME = models.CharField(max_length=100, null=True,
blank=True)

    MOBILE = models.CharField(max_length=15, null=True, blank=True)

    TOTALPRICE = models.IntegerField(null=True, blank=True)



    def __str__(self):

        return f"{self.CUSTOMERNAME} - Rs.{self.TOTALPRICE}"
```

10.2 Django Models – Backend/models.py

```
from django.db import models



class roomtypedb(models.Model):

    ROOMTYPE = models.CharField(max_length=100, null=True,
blank=True)

    ROOMTYPEIMAGE = models.ImageField(
```

```
        upload_to="roomtypeimages", null=True, blank=True

    )

    DESCRIPTION = models.CharField(max_length=100, null=True,
blank=True)



    def __str__(self):

        return self.ROOMTYPE or "No Type"



class roomnamedb(models.Model):

    ROOMTYPE = models.ForeignKey(

        roomtypedb, on_delete=models.CASCADE,

        null=True, blank=True, related_name='rooms'

    )

    ROOMNAME = models.CharField(max_length=100, null=True,
blank=True)

    ROOMIMAGE = models.ImageField(

        upload_to="roomimage", null=True, blank=True

    )

    ROOMPRICE = models.IntegerField(null=True, blank=True)

    ROOMDESCRIPTION = models.CharField(max_length=100, null=True,
blank=True)
```

```
    def __str__(self):
        return self.ROOMNAME or "No Room Name"


class staffdb(models.Model):
    STAFFNAME = models.CharField(max_length=100, null=True, blank=True)
    STAFFDESIGNATION = models.CharField(max_length=100, null=True, blank=True)
    STAFFACEBOOK = models.CharField(max_length=100, null=True, blank=True)
    STAFFINSTA = models.CharField(max_length=100, null=True, blank=True)
    STAFFIMAGE = models.ImageField(
        upload_to="staffimage", null=True, blank=True
    )


    def __str__(self):
        return self.STAFFNAME or "No Name"


from .chatbot_models import ChatbotResponse, ChatbotConversation, Notification
```

10.3 Booking with Overlap Detection – webapp/views.py

```
def save_booking_page(request, CONTACTEMAIL=None):

    if request.method == "POST":

        na = request.POST.get('bname')

        em = request.POST.get('bemail')

        chn = request.POST.get('bcheckin')

        cho = request.POST.get('bcheckout')

        ta = request.POST.get('btotaladults')

        tc = request.POST.get('btotalchilds')

        sr_id = request.POST.get('bselectroom')

        sre = request.POST.get('bspecialrequest')

        tp = request.POST.get('btotalprice')



        try:

            room_obj = roomnamedb.objects.get(id=sr_id)

        except roomnamedb.DoesNotExist:

            messages.error(request, "Invalid room selection.")
```

```
            return redirect('save_room_page')



        # Check for overlapping bookings using date-range logic

        overlapping_bookings = bookingdb.objects.filter(

            SELECTROOM=room_obj

        ).filter(

            CHECKIN__lte=cho,    # Existing check-in is before new
check-out

            CHECKOUT__gte=chn    # Existing check-out is after new
check-in

        )



        if overlapping_bookings.exists():

            messages.error(request,

                "The selected room is already booked for the
specified dates.")

            return redirect('save_room_page')

        else:

            customer_obj = None

            if 'USERNAME' in request.session:

                try:
```

```
                customer_obj = Registerdb.objects.get(
                    USERNAME=request.session['USERNAME']
                )
            except Registerdb.DoesNotExist:
                pass


        obj = bookingdb(
            CUSTOMER=customer_obj,
            CUSTOMERNAME=na,
            CONTACTEMAIL=em,
            CHECKIN=chn,
            CHECKOUT=cho,
            TOTALADULTS=int(ta) if ta else None,
            TOTALCHILDS=int(tc) if tc else None,
            SELECTROOM=room_obj,
            SPECIALREQUEST=sre,
            TOTALPRICE=int(tp) if tp else None
        )
        obj.save()
```

```
            messages.success(request, "Room booked successfully.")



            subject = "Congratulations - Room Booked"

            message = (f"Dear customer, you have booked a room "

                       f"successfully. Thank you & have a nice
day!")

            send_mail(subject, message, EMAIL_HOST_USER,

                      [obj.CONTACTEMAIL], fail_silently=True)

            return redirect('save_room_page')



        return redirect('save_room_page')
```

10.4 Chatbot Engine – Backend/chatbot_views.py

```
@csrf_exempt

@require_http_methods(["POST"])

def chatbot_query(request):

    try:
```

```
        data = json.loads(request.body)

        user_message = data.get('message', '').strip().lower()

        username = data.get('username', 'Guest')



        if not user_message:

            return JsonResponse({'error': 'Empty message'},
status=400)



        response_data = generate_chatbot_response(user_message,
username)



        ChatbotConversation.objects.create(

            username=username,

            user_message=data.get('message'),

            bot_response=response_data['response'],

            query_type=response_data.get('query_type', 'general')

        )



        return JsonResponse({

            'response': response_data['response'],
```

```
            'query_type': response_data.get('query_type',
'general'),
            'suggestions': response_data.get('suggestions', []),
            'data': response_data.get('data', [])
        })
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON'}, status=400)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)


def generate_chatbot_response(user_message, username):
    response = {
        'response': '', 'query_type': 'general',
        'suggestions': [], 'data': []
    }
    room_keywords = ['room', 'price', 'availability', 'luxury',
                     'deluxe', 'suite', 'bed', 'rooms']
    staff_keywords = ['staff', 'team', 'manager', 'receptionist',
                      'housekeeper', 'chef']
    booking_keywords = ['booking', 'reserve', 'book',
```

```
'reservation',
                        'check-in', 'check-out', 'cancel']



    if any(kw in user_message for kw in room_keywords):
        return handle_room_query(user_message, username, response)
    elif any(kw in user_message for kw in staff_keywords):
        return handle_staff_query(user_message, username, response)
    elif any(kw in user_message for kw in booking_keywords):
        return handle_booking_query(user_message, username,
response)
    else:
        return handle_general_query(user_message, username,
response)
```

10.5 Payment Integration – webapp/views.py

```
def payment_page(request):
    customer = Totaldb.objects.order_by('-id').first()
    payy = customer.TOTALPRICE
    amount = int(payy * 100)  # Razorpay expects amount in paise
```

```
    payy_str = str(amount)



    if request.method == "POST":

        order_currency = 'INR'

        client = razorpay.Client(

            auth=('rzp_test_klfWwvjaFXjXmC',
's9P7dwOwYckK352FfRJOXIRV')

        )

        payment = client.order.create({

            'amount': amount,

            'currency': order_currency,

            'payment_capture': '1'

        })



    return render(request, "payment.html",

                  {'customer': customer, 'payy_str': payy_str})
```

10.6 OTP-Based Password Reset – webapp/views.py

```
def generate_otp(request):

    otp = random.randint(10000, 55555)

    return otp



def reset_password_email_verification(request):

    if request.method == "POST":

        email = request.POST.get('email')

        if Registerdb.objects.filter(EMAIL=email).exists():

            user = Registerdb.objects.get(EMAIL=email)

            u_otp = generate_otp(request)

            request.session['otp'] = u_otp

            subject = "Forgot Password"

            message = (f"Dear user, OTP for resetting your password
"

                       f"is: {u_otp}. Thank you, Hoteliers")

            send_mail(subject, message, EMAIL_HOST_USER,
```

```
                    [email], fail_silently=True)

            context = {

                'message': "An OTP sent to your registered email.",

                'user': user,

            }

            return render(request, "16password_reset_otp.html",
context)

        else:

            return render(request, "15password_reset_email.html",

                          {'error': "Invalid email id"})



def passwordReset_verify_otp(request, user_id):

    user = Registerdb.objects.get(id=user_id)

    if request.method == "POST":

        u_otp = str(request.POST.get('otp')).strip()

        s_otp = str(request.session.get('otp')).strip()

        if u_otp == s_otp:

            return render(request, "17reset_password.html",

                          {'user': user, 'username':
user.USERNAME})
```

```
        else:
            return render(request, "16password_reset_otp.html",
                          {'error': "OTP does not match!", 'user':
user})



def reset_password(request, username):
    if request.method == "POST":
        user = Registerdb.objects.get(USERNAME=username)
        password1 = request.POST.get('password1')
        password2 = request.POST.get('password2')
        if password1 == password2:
            Registerdb.objects.filter(
                USERNAME=username
            ).update(PASSWORD=password1)
            messages.success(request, "Password reset
successfully!")
            send_mail("Password Changed",
                      "Your password has been changed.",
                      EMAIL_HOST_USER, [user.EMAIL],
                      fail_silently=True)
```

```
            return redirect("user_login_page")

        else:

            return render(request, "17reset_password.html",

                          {'error': "Passwords do not match!",

                           'username': username})
```

11. Testing

11.1 Testing Strategy

Testing is a critical phase of the Software Development Life Cycle that ensures the software meets the specified requirements and is free from defects. The testing strategy for this project involved four levels of testing, each with a specific focus:

1. **Unit Testing:** Testing individual functions, views, and model methods in isolation to verify that each unit produces the correct output for a given input.
2. **Integration Testing:** Testing the interaction between two or more connected modules to verify that data flows correctly from one module to another.
3. **System Testing:** Testing the complete, integrated system as a whole to verify that it meets all functional and non-functional requirements from the user's perspective.
4. **User Acceptance Testing (UAT):** Testing by potential end-users (hotel staff and simulated customers) to verify that the system is usable, intuitive, and meets real-world expectations.

For each test case, the test ID, description, input, expected result, actual result, and pass/fail status are documented.

11.2 Unit Testing

Test ID	Test Case Description	Input	Expected Result	Status
UT-01	User registration with valid data	Username: "john", Email: "john@mail.com", Password: "pass123", Confirm: "pass123"	New record created in Registerdb; redirect to sign-in page with success message	**Pass ✓**
UT-02	User login with valid credentials	Username: "john", Password: "pass123"	Session created with USERNAME="john"; redirect to home page	**Pass ✓**
UT-03	User login with wrong password	Username: "john", Password: "wrong"	Warning message "Signin failed"; redirect to sign-in page; no session	**Pass ✓**

			created	
			Session destroyed;	
UT-04	User logout	Click "Sign Out" while logged in	redirect to home page with success message	**Pass ✓**
UT-05	Add room type with image	Name: "Deluxe", Description: "Luxury room", Image: deluxe.jpg	Record saved in roomtypedb; success message displayed	**Pass ✓**
UT-06	Add room with FK to room type	Type: "Deluxe" (dropdown), Name: "Room 101", Price: 5000, Image: room101.jpg	Record saved in roomnamedb with ROOMTYPE FK pointing to correct type	**Pass ✓**
UT-07	Edit room type	Change name from "Deluxe" to "Super Deluxe"	Record updated; rooms still linked correctly	**Pass ✓**
UT-	Delete room	Delete "Standard"	Room type and its 2	**Pass ✓**

08	type	type with 2 rooms	rooms deleted (CASCADE)	
UT-09	Add staff member	Name: "Priya", Designation: "Receptionist", Image: priya.jpg	Record saved in staffdb; displayed on staff page	**Pass ✓**
UT-10	Save customer contact message	Name: "Guest", Email: "guest@mail.com", Subject: "Enquiry"	Record saved in customercontactdb; success message	**Pass ✓**
UT-11	Book room – available (no overlap)	Room: 101, Checkin: 2026-03-20, Checkout: 2026-03-22	Booking saved; confirmation email sent; redirect to cart	**Pass ✓**
UT-12	Book room – overlapping dates	Room: 101, Checkin: 2026-03-21, Checkout: 2026-03-23	Error message "Room already booked for these dates"; booking not	**Pass ✓**

		(overlaps UT-11)	saved	
UT-13	Book room – adjacent dates (no overlap)	Room: 101, Checkin: 2026-03-22, Checkout: 2026-03-24	Booking saved (checkout of UT-11 = checkin of this; boundary is acceptable)	**Pass ✓**
UT-14	Cart subtotal calculation	Two bookings: ₹3000 + ₹5000	Subtotal: ₹8000, Tax: 10 (>5000), Total: ₹8010	**Pass ✓**
UT-15	Delete booking from cart	Delete booking ID=5	Booking removed; cart recalculated; warning message	**Pass ✓**
UT-16	OTP generation	Call generate_otp()	Random integer between 10000 and 55555	**Pass ✓**
UT-17	OTP email dispatch	Valid email "john@mail.com"	OTP stored in session; email sent to john@mail.com	**Pass ✓**
UT-	OTP	Entered OTP	New password form	**Pass ✓**

18	verification – correct OTP	matches session OTP	displayed	
UT-19	OTP verification – incorrect	Entered OTP does not match	Error "OTP does not match"; OTP form redisplayed	**Pass ✓**
UT-20	Password reset – matching passwords	Password1: "new123", Password2: "new123"	Password updated; notification email sent; redirect to sign-in	**Pass ✓**
UT-21	Password reset – mismatched passwords	Password1: "new123", Password2: "new456"	Error "Passwords do not match"; form redisplayed	**Pass ✓**
UT-22	Chatbot – room query	"Show me luxury rooms"	Response with room cards (type, name, price, image)	**Pass ✓**
UT-23	Chatbot – staff query	"Tell me about the staff"	Response with staff count and details	**Pass ✓**
UT-	Chatbot –	"How do I cancel	Response with	**Pass ✓**

24	booking query	my booking?"	cancellation information and suggestions	
UT-25	Chatbot – general greeting	"Hello"	Response: "Hello! 👋 Welcome to our hotel..."	**Pass ✓**
UT-26	Rate chatbot response	conversation_id=1, rating=5	Rating saved; success message returned	**Pass ✓**
UT-27	Advanced search API – by price range	min_price=2000, max_price=5000	JSON array of rooms with price between 2000–5000	**Pass ✓**
UT-28	Advanced search API – by room type	room_type="Deluxe"	JSON array of rooms of type "Deluxe" only	**Pass ✓**
UT-29	Advanced search API – by date availability	check_in=2026-03-20, check_out=2026-03-22	JSON array excluding rooms booked for those dates	**Pass ✓**

UT-30	PDF generation	Logged-in user with bookings	PDF file downloaded with booking details	**Pass ✓**

11.3 Integration Testing

Test ID	Test Case	Modules Involved	Expected Result	Status
IT-01	Registration → Login → Booking	Module 1 → Module 5	User registers → signs in (session created) → books room → CUSTOMER FK links booking to Registerdb record	**Pass ✓**
IT-02	Booking → Payment Processing	Module 5 → Module 6	Booking saved → cart shows subtotal/tax/total → Totaldb record created with BOOKING FK → Razorpay order created	**Pass ✓**
IT-	Room Type →	Module 2	Room type created →	**Pass ✓**

			room added with FK →	
03	Room → Search Results	→ Module 5	advanced search returns room with correct type name	
IT-04	Contact Form → Admin View	Module 4 → Admin Backend	Customer submits contact form → record appears on admin contact messages page	**Pass ✓**
IT-05	Chatbot → Conversation Storage → Admin	Module 8 → Django Admin	User sends chat → conversation saved in ChatbotConversation → visible in Django Admin	**Pass ✓**
IT-06	Booking → Email Notification	Module 5 → Module 7	Booking saved → confirmation email sent to customer's email address	**Pass ✓**
IT-07	Password Reset → OTP Email → Password Update	Module 1 → Module 7	Email entered → OTP generated and emailed → OTP verified →	**Pass ✓**

			password updated → notification email sent	
IT-08	Room Type Delete → Cascade to Rooms → Cascade to Bookings	Module 2 → Module 5	Deleting room type cascades to rooms and their bookings (referential integrity maintained)	**Pass ✓**
IT-09	Chatbot Room Query → Database Lookup → Card Rendering	Module 8 → Module 2	User asks about rooms → chatbot queries roomnamedb → returns room cards with images from media folder	**Pass ✓**
IT-10	Search with Date Filter → Overlap Check	Module 5 (Search + Booking)	Search excludes rooms with bookings overlapping the requested dates	**Pass ✓**

11.4 System Testing

Test	Scenario	Steps	Expected	Status

ID			Outcome	
ST-01	Complete customer journey	Register → Sign in → Browse rooms → Advanced search → Select room → Book → View cart → Enter payment details → Pay via Razorpay → Sign out	All steps complete successfully; data persisted in all tables; email received	**Pass ✓**
ST-02	Complete admin management cycle	Admin login → Add room type → Add room → Add staff → View bookings → View payments → View contacts → Django Admin chatbot → Logout	All CRUD operations successful; data visible on all pages	**Pass ✓**
ST-	Chatbot end-to-	Open chatbot →	All query types	**Pass ✓**

03	end	Room query → Get room cards → Staff query → Booking query → General greeting → Rate responses	return correct responses; conversations saved; ratings stored	
ST-04	Password reset end-to-end	Forgot password → Enter email → Receive OTP → Enter OTP → Set new password → Login with new password	OTP verified; password updated; old password rejected; new password works	**Pass ✓**
ST-05	Responsive design – Desktop	Open all pages at 1920×1080 resolution	Full layout; proper spacing; all features visible and functional	**Pass ✓**
ST-	Responsive	Open all pages at	Layout adapts; no	**Pass ✓**

			horizontal overflow;	
06	design – Tablet	768×1024 resolution	navigation collapses to hamburger menu	
ST-07	Responsive design – Mobile	Open all pages at 375×667 resolution	Single-column layout; touch-friendly buttons; all features accessible	**Pass ✓**
ST-08	Cross-browser – Chrome	All features in Chrome 120+	All features work correctly	**Pass ✓**
ST-09	Cross-browser – Edge	All features in Edge 120+	All features work correctly	**Pass ✓**
ST-10	Cross-browser – Firefox	All features in Firefox 120+	All features work correctly	**Pass ✓**

11.5 User Acceptance Testing (UAT)

User Acceptance Testing was conducted by allowing potential

end-users (friends and family members acting as hotel staff and customers) to use the system and provide feedback.

Test ID	User Role	Task Given	User Feedback	Status
UAT-01	Customer	Register, browse rooms, book a room, and make payment	"The process was smooth. I could easily find and book a room."	**Accepted** ✓
UAT-02	Customer	Use the chatbot to find room information	"The chatbot answered my questions about rooms quickly. The room cards were helpful."	**Accepted** ✓
UAT-03	Customer	Reset password using email OTP	"I received the OTP promptly and was able to reset my password without issues."	**Accepted** ✓
UAT-04	Admin	Add room types, rooms, and	"The admin panel is straightforward.	**Accepted**

		staff; view bookings and payments	Adding and managing data is easy."	✓
UAT-05	Customer	Use advanced search to find rooms by price range and date	"The search results appeared instantly. I liked that it showed availability."	**Accepted** ✓

11.6 Debugging and Bug Fixing

During the testing phase, several bugs and issues were identified and resolved:

Bug ID	Description	Root Cause	Fix Applied
BUG-01	Double booking allowed for same room and dates	Simple equality check on dates instead of date-range overlap logic	Implemented proper overlap detection: `CHECKIN__lte=checkout, CHECKOUT__gte=checkin`
BUG-	Room type	Image field not	Added `enctype="multipart/form-`

02	image not displaying after edit	included in the edit form's enctype	`data"` to the edit form
BUG-03	Chatbot returned empty response for some queries	Keyword matching was case-sensitive	Converted user message to lowercase before comparison
BUG-04	ForeignKey assignment error when adding room	Passing string ID instead of model object	Used `roomtypedb.objects.get(id=type_id)` to get the object before assignment
BUG-05	OTP comparison failed even with correct OTP	Whitespace in pasted OTP string	Added `.strip()` to both user input and session value before comparison
BUG-06	Payment amount incorrect in	Amount not multiplied by 100 (Razorpay uses	Changed to `amount = int(price * 100)`

	Razorpay	paise)	
BUG-07	Staff page crash when staff has no image	Template accessing `.url` on null ImageField	Added conditional check in chatbot: `if room.ROOMIMAGE else None`
BUG-08	Search by room type returned all rooms	Filter used wrong field name	Changed to `ROOMTYPE__ROOMTYPE__icontains` for FK lookup

12. Validations Performed

The following validations are implemented throughout the system to ensure data integrity, prevent erroneous input, and maintain a consistent user experience:

1. **Unique Customer ID:** Every registered user has a unique, auto-generated Customer ID (primary key) in the USER_MASTER (Registerdb) table, ensuring that no two users share the same identifier.
2. **Mandatory Fields:** All compulsory fields such as username, email, password, room selection, check-in date, check-out date, and total price must not be left blank in the signup and booking forms. The server-side view functions check for the presence of these values before saving records to the database.
3. **Password Masking:** In password fields, characters typed by the user are masked with dot symbols (●) so that the actual password is not visible on the screen, preventing shoulder-surfing attacks.
4. **Numeric Field Validation:** The system does not accept alphabetic characters or negative values where only numeric amounts are required, such as room price, number of adults, number of children, and total amount. The Django model defines these fields as `IntegerField`, and the HTML input uses `type="number"` with appropriate min/max attributes.
5. **Date Format Validation:** Proper date format validation is applied for CHECKIN and CHECKOUT fields. The Django

model uses `DateField`, and the HTML input uses `type="date"` to enforce the correct format. The system also validates that the checkout date is after the check-in date.

6. **Date-Range Overlap Detection:** A booking is not saved if the selected room is already booked for any overlapping date range. The system queries existing bookings where `CHECKIN ≤ new_checkout AND CHECKOUT ≥ new_checkin`. If any such record exists, the booking is rejected with an error message, ensuring no double booking occurs.
7. **Referential Integrity:** Master information such as room types and rooms that are linked to existing bookings are protected by Django's `on_delete` constraints. Cascade deletion ensures that orphan records are not left in the database, and SET_NULL ensures that deleting a user does not delete their booking history.
8. **Authentication Gate:** Any unauthorized person cannot enter into the system because each user is validated using username and password at login. Admin pages are restricted to users authenticated via Django's built-in `authenticate()` function. Customer pages that require login (booking, cart, payment) check for the presence of `USERNAME` in the session.
9. **Email Format Validation:** Email fields use Django's `EmailField`, which validates that the entered value follows the standard email format (user@domain.com). This prevents invalid email addresses from being stored in the database.
10. **Image File Validation:** Image upload fields use Django's `ImageField`, which validates that the uploaded file is a valid image format (JPEG, PNG, AVIF, etc.) and rejects non-image files.
11. **CSRF Token Validation:** Every form submission is validated for the presence of a valid CSRF token. Django's

CSRF middleware rejects any POST request that does not include the correct token, preventing cross-site request forgery attacks.

12. **OTP Validation:** During password reset, the entered OTP is compared (after stripping whitespace) with the OTP stored in the session. If they do not match, the reset process is halted and an error message is displayed.

13. Input and Output Screens

The following table lists all the key input and output screens of the system. Original screenshots of each screen should be captured from the running application and attached in the printed report.

#	Screen Name	Type	Description
1	Home Page	Output	The landing page of the website displaying room type category cards with images, a navigation bar, and the floating chatbot widget. Each room type card shows the type name, description, and image, with a link to view rooms of that category.
2	User Sign-Up Page	Input	Registration form with four fields: username, email address, password, and confirm password. Each field has appropriate validation. A "Sign Up" button submits the form, and a link

			redirects to the sign-in page for existing users.
3	User Sign-In Page	Input	Login form with two fields: username and password. Includes a "Sign In" button and a "Forgot Password?" link for password recovery.
4	Forgot Password – Email Verification	Input	Single email input field where the user enters their registered email address to receive an OTP for password reset.
5	Forgot Password – OTP Entry	Input	OTP input field where the user enters the OTP received via email. Includes a message confirming that the OTP has been sent.
6	Forgot Password – New Password	Input	Two password fields (new password and confirm new password) for setting the new password after OTP verification.
7	About / Our	Output	Displays staff member cards with photo,

	Team Page		name, designation, and social media links (Facebook, Instagram).
8	Services Page	Output	Lists the hotel's services and amenities with icons and descriptions.
9	Room Categories Page	Output	Displays all room type categories as cards with images and "View Rooms" buttons.
10	Filtered Rooms Page	Output	Shows all rooms of a selected type with room name, image, price, description, and "Book Now" button.
11	Advanced Search Page	Input/ Output	Left panel with filter inputs (room name, room type dropdown, min price, max price, check-in date, check-out date, search button). Right panel displays AJAX-loaded room result cards.
12	Booking Form Page	Input	Form fields: customer name, email, check-in date, check-out date, number of adults, number of children, selected room (hidden), special request

			(textarea), total price (calculated). "Confirm Booking" button.
13	Booking Cart / Saved Rooms Page	Output	Table of booked rooms showing room name, dates, price, and a delete (×) button. Below the table: subtotal, tax, and total. "Proceed to Payment" button.
14	Payment Page (Razorpay)	Input/ Output	Shows customer name, mobile number input, and total amount. Razorpay payment button triggers the payment popup with card/UPI/net banking options.
15	Customer Contact Form	Input	Five-field form: name, phone number, email, subject, and message (textarea). "Send Message" button.
16	Chatbot Widget	Input/ Output	Floating chat icon that opens a chat window. Message input at the bottom. Chat history with user messages (right-aligned) and bot responses (left-aligned). Suggestion chips below bot

			responses. Room cards displayed for room queries.
17	Admin Login Page	Input	Username and password fields for admin authentication using Django's auth system.
18	Admin – Add Room Type	Input	Form with room type name, description, and image upload fields.
19	Admin – Display Room Types	Output	Table listing all room types with columns: ID, name, image (thumbnail), description, edit button, delete button.
20	Admin – Add Room	Input	Form with room type (dropdown FK), room name, price, description, and image upload.
21	Admin – Display Rooms	Output	Table listing all rooms with columns: ID, room type, room name, image, price, description, edit, delete.
22	Admin – Add Staff	Input	Form with staff name, designation, Facebook URL, Instagram URL, and image upload.

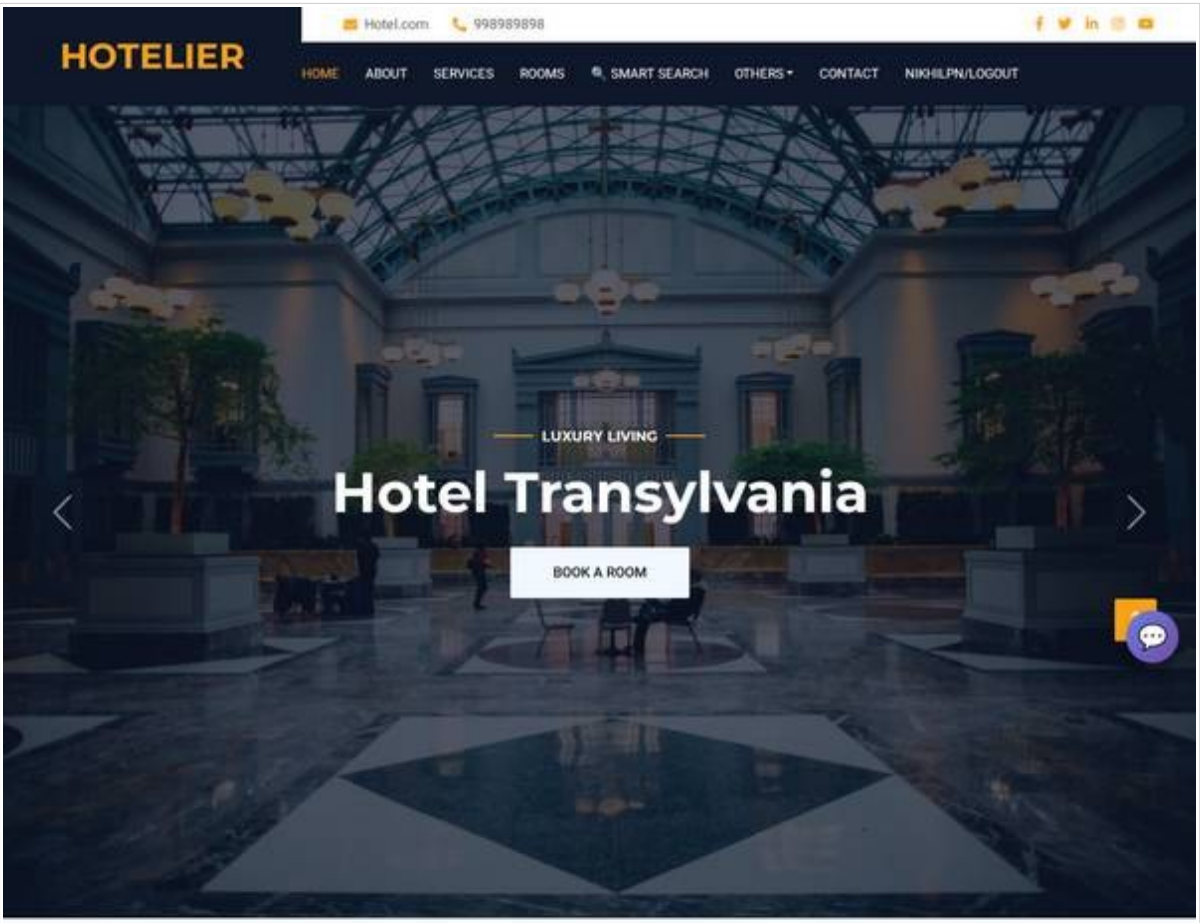
23	Admin – Display Staff	Output	Table listing all staff with columns: ID, name, designation, social links, image, edit, delete.
24	Admin – Customer Contact Messages	Output	Table listing all contact messages with name, phone, email, subject, message, and delete button.
25	Admin – Booked Details	Output	Table listing all bookings with customer name, email, room, check-in, check-out, adults, children, price.
26	Admin – Payment Totals	Output	Table listing payment records with customer name, mobile, total amount, and linked booking.

Important Note: Attach original screenshots of each screen in the printed hard-bound copy of the project report. Do not use xerox copies of the screenshots. Each screenshot should be clearly labelled with a figure number and caption.

13.1 Screenshots of Key Screens

The following screenshots were captured from the running application and demonstrate the key input and output screens of

the Hotel Room Booking System.



ABOUT US

Welcome to HOTELIER

Sure! Here's a concise and engaging bio for a hotel: Welcome to megha, your premier destination for comfort and luxury! Nestled in the heart of [Location], we offer a perfect blend of modern amenities and timeless elegance. Our spacious rooms feature stunning views, plush bedding, and all the essentials for a relaxing stay. Enjoy our on-site dining options, rejuvenate at our spa, or unwind by the pool. Whether you're here for business or leisure, our dedicated staff is committed to making your stay unforgettable. Experience the charm of Thrissur with us!

EXPLORE MORE

Figure 13.1 – Home Page: The landing page of the website displaying room type category cards, navigation bar, and chatbot widget.

Figure 13.2 – Admin Dashboard: Admin backend dashboard showing the sidebar navigation, statistics cards, and main content area.

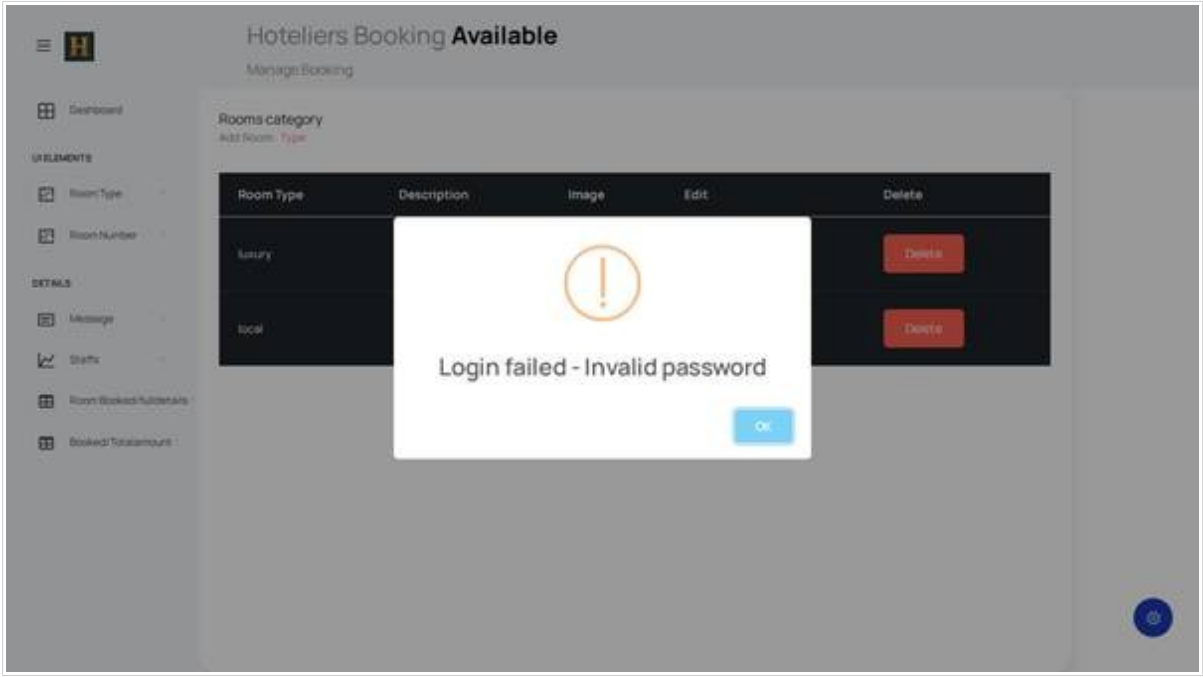


Figure 13.3 – Admin – Room Type List: Table listing all room types with columns for ID, name, image, description, and action buttons.

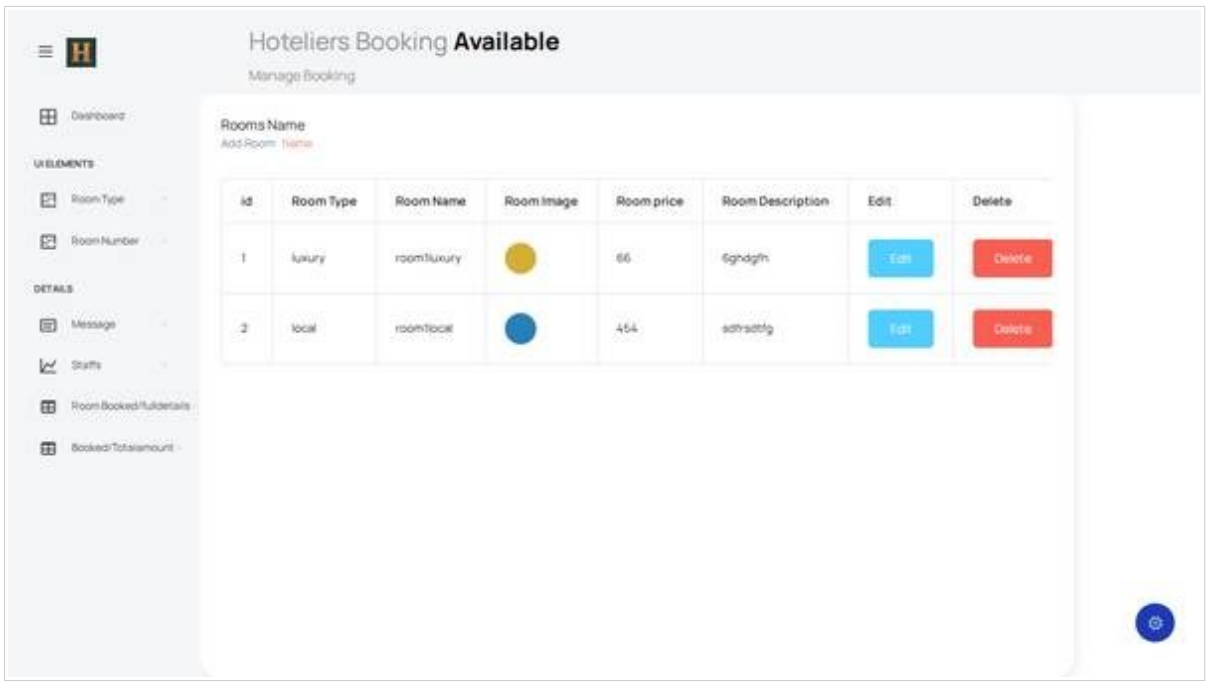


Figure 13.4 – Admin – Room List: Table listing all rooms with room type, name, image, price, description, and edit/delete actions.

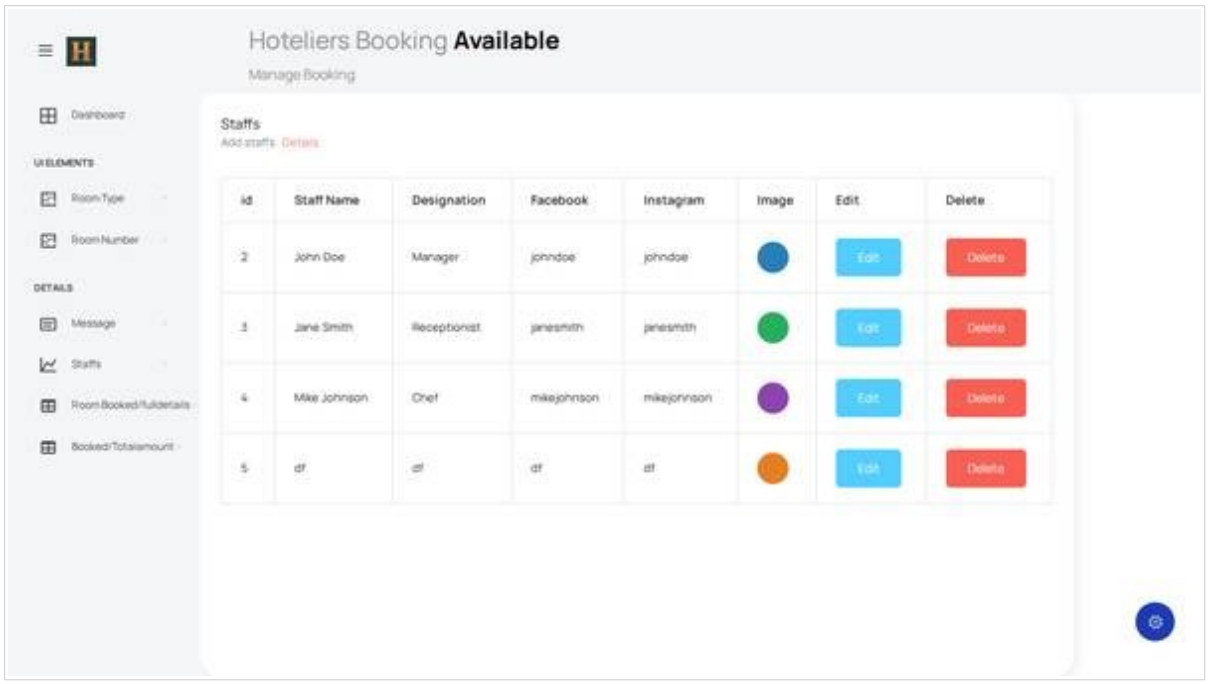


Figure 13.5 – Admin – Staff List: Table listing staff members with name, designation, social links, image, and edit/delete buttons.

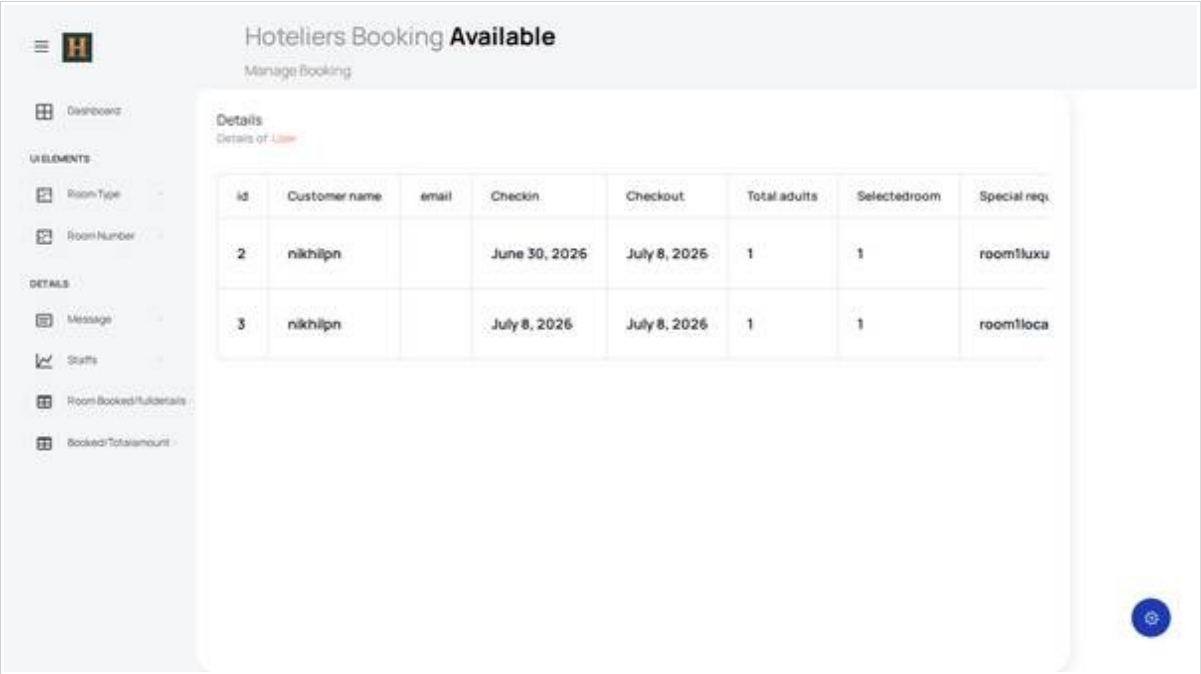


Figure 13.6 – Admin – Bookings: Booking records table showing customer name, email, room, dates, guests, and total price.

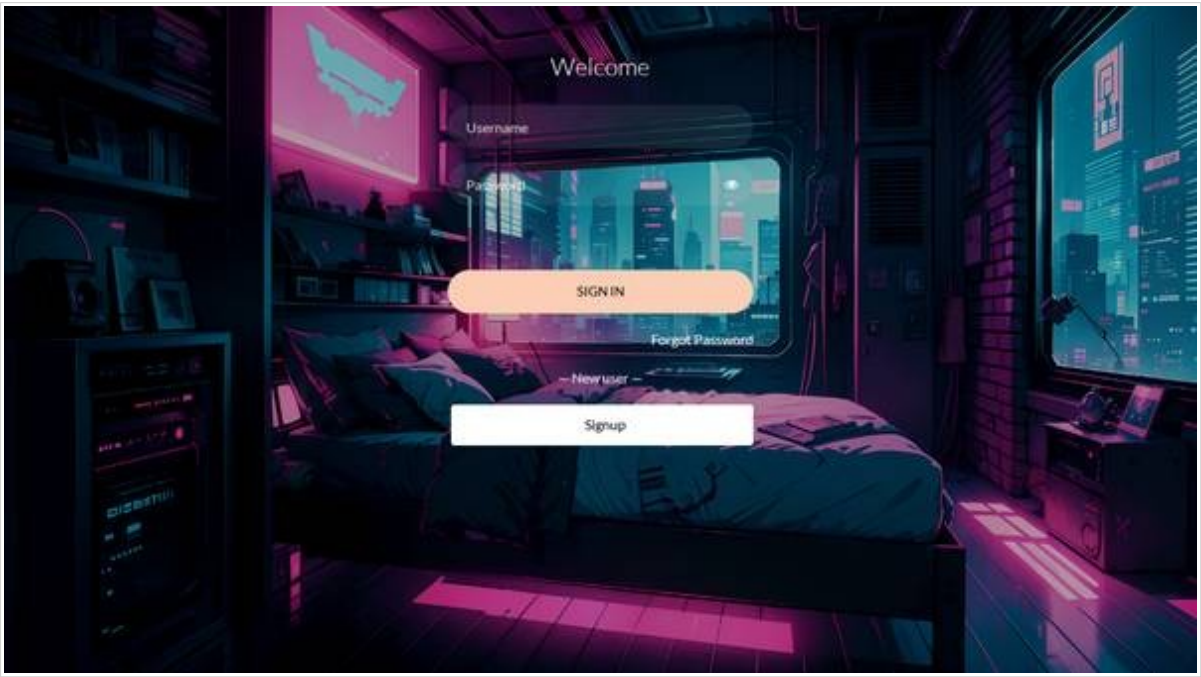


Figure 13.7 – User Login Page: Sign-in form with username and password fields, sign-in button, and forgot password link.


HOTELIER
Hotel.com
998989898
HOME
ABOUT
SERVICES
ROOMS
OTHERS
CONTACT
NIKHILPN/LOGOUT
Rooms
HOME / PAGES / ROOMS
OUR ROOMS
Explore Our ROOMS
luxury
1 Bed
1 Bath
Wifi
swrsdsd
VIEW DETAIL
local
1 Bed
1 Bath
Wifi
fef
VIEW DETAIL

Figure 13.8 – Rooms Listing Page: Room category cards with images and 'View Rooms' buttons for browsing rooms.

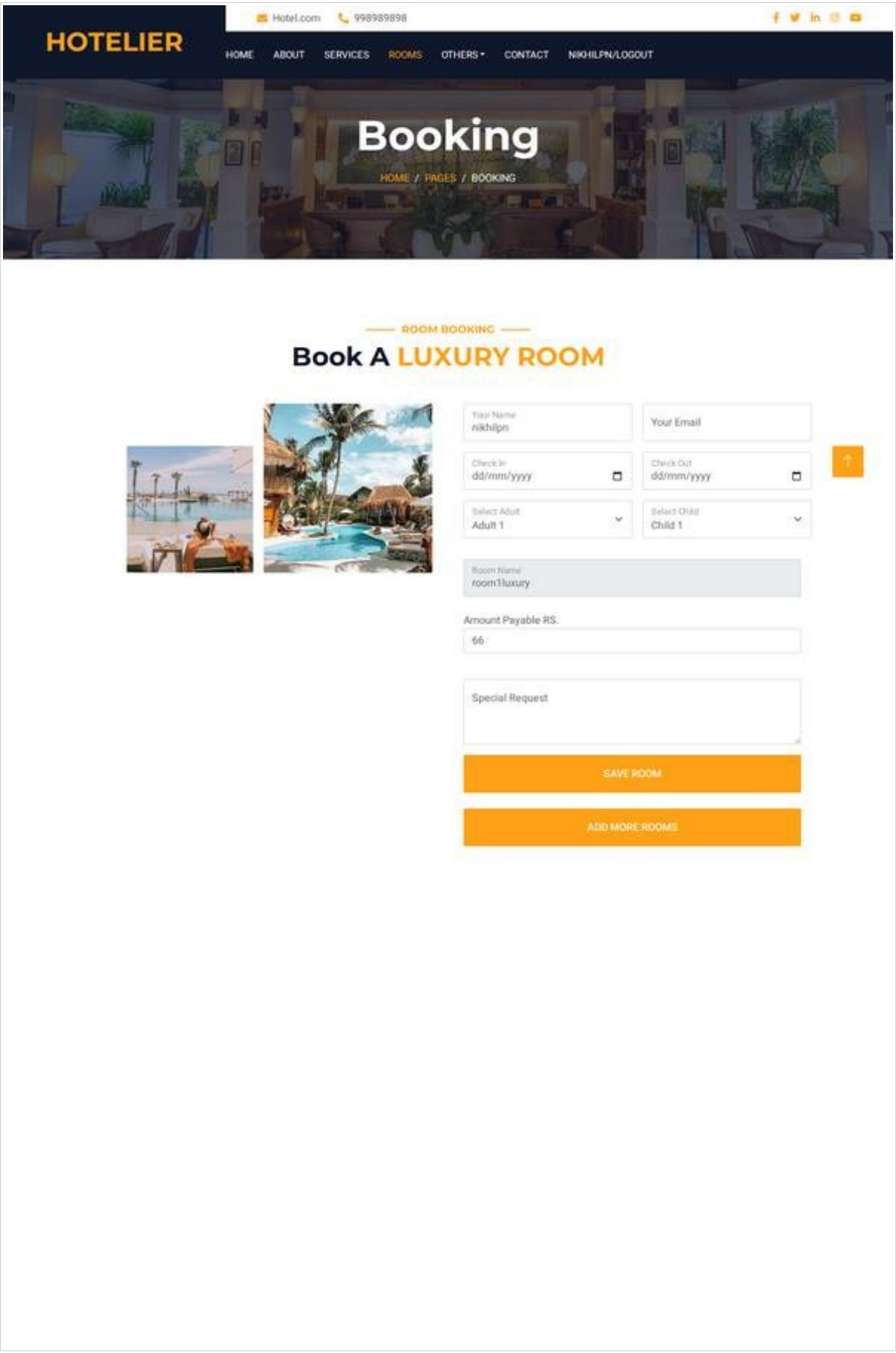

HOTELIER
Hotel.com
998989898
HOME
ABOUT
SERVICES
ROOMS
OTHERS
CONTACT
NIKHILPN/LOGOUT
Booking
HOME / PAGES / BOOKING
ROOM BOOKING
Book A LUXURY ROOM
Your Name
nikhilpn
Your Email
Check In
dd/mm/yyyy
Check Out
dd/mm/yyyy
Select Adult
Adult 1
Select Child
Child 1
Room Name
roomTluxury
Amount Payable RS.
66
Special Request
SAVE ROOM
ADD MORE ROOMS

Figure 13.9 – Booking Form: Booking input form with customer details, check-in/out dates, guest count, and price.

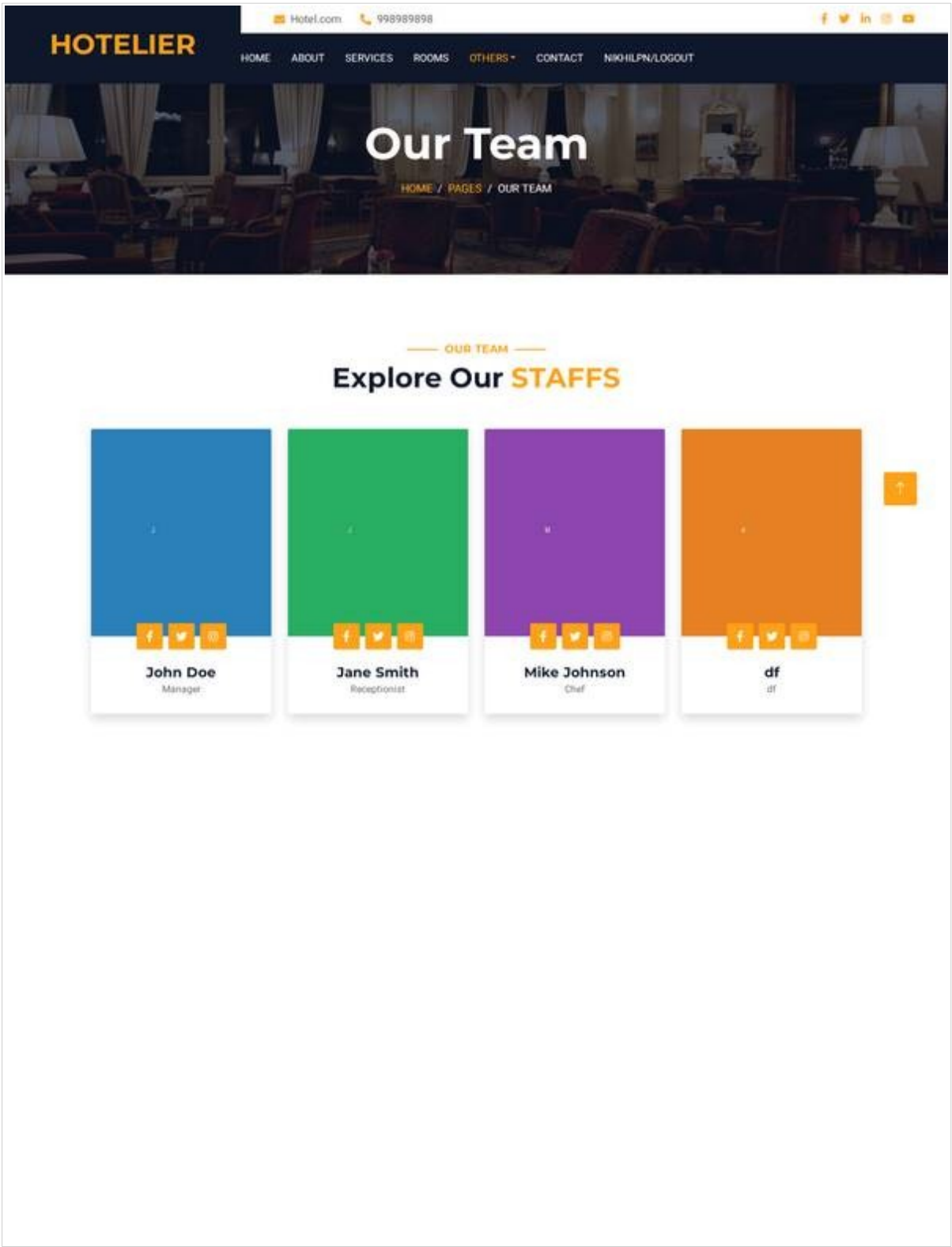


Figure 13.10 – About / Our Team Page: Staff profile cards with photos, names, designations, and social media links.

ABOUT US

Welcome to HOTELIER

hi erat elitr rebum at clita. Diam dolor diam ipsum sit. Aliqu diam amet diam et eos. Clita erat ipsum et lorem et sit, sed stet lorem sit clita duo justo magna dolore erat amet

858 Rooms

858 Staffs

858 Clients

EXPLORE MORE

Figure 13.11 – About Page (Hotel Info): Hotel information page with description, services overview, and team introduction.

14. Report Generation

The system generates the following reports for administrative use, customer reference, and record-keeping:

Report Name	Description	Format
Customer Booking Report	Displays details of customers who have booked rooms including customer name, email, mobile, selected room (with room type), check-in date, check-out date, number of adults, number of children, special requests, and total price. This report helps the hotel administration track all active and past bookings.	HTML (on-screen) PDF (downloadable)
Room Booking Summary	A summarized list of all bookings for each room showing room type, room name, booking dates, occupancy	HTML (on-screen)

	(adults + children), and total revenue per room. This report helps the hotel analyze room usage, identify popular rooms, and plan inventory.	
Payment / Revenue Report	Payment details from the Totaldb table including customer name, mobile number, booking reference (FK), and total amount paid. This report supports financial tracking, reconciliation with Razorpay, and revenue analysis.	HTML (on-screen)
Customer Contact Report	All messages submitted through the customer contact form with customer name, email, phone, subject, and full message text. This report helps the hotel staff follow up on guest enquiries and complaints.	HTML (on-screen)
Staff Details Report	Staff information including name, designation, social media links, and	HTML (on-screen)

photo. This report is used for internal

HR records and for generating the

"Our Team" section on the website.

The PDF report generation uses the `xhtml2pdf` library, which converts an HTML template (rendered using Django's template engine) into a downloadable PDF document. The process involves:

1. Preparing the context data (booking details, customer name, etc.)
2. Rendering an HTML template to a string using `render_to_string()`
3. Creating an HttpResponse with `content_type='application/pdf'`
4. Setting the `Content-Disposition` header for file download
5. Calling `pisa.CreatePDF()` to convert the HTML string to PDF
6. Returning the PDF response to the browser

15. Implementation of Security

Security is a critical aspect of any web application, especially one that handles sensitive data such as customer personal information, booking details, and financial transactions. The following security measures are implemented in this project:

1. **User Authentication:** Customer login is verified by checking the entered username and password against the Registerdb table. Upon successful login, the username is stored in the Django session. Pages that require authentication (booking, cart, payment, profile) check for the presence of `request.session['USERNAME']` before allowing access. If the session variable is not present, the user is redirected to the sign-in page.
2. **Admin Authentication:** Admin backend access is protected using Django's built-in authentication system. The admin login view uses `authenticate(username, password)` and `login(request, user)` functions from `django.contrib.auth`. Only authenticated admin users can access admin pages.
3. **CSRF (Cross-Site Request Forgery) Protection:** Django's CSRF middleware (`django.middleware.csrf.CsrfViewMiddleware`) is enabled in the project settings. Every form in the application includes the template tag, which generates a hidden CSRF token. All POST requests are validated for the presence of a valid CSRF token, preventing attackers from submitting forms on behalf of authenticated users.
4. **SQL Injection Prevention:** All database queries are performed through Django's ORM (Object-Relational

Mapper), which uses parameterised queries internally. This prevents SQL injection attacks because user input is never concatenated directly into SQL strings. For example, `Registerdb.objects.filter(USERNAME=un, PASSWORD=pswd)` generates a parameterised SQL query.

5. **XSS (Cross-Site Scripting) Protection:** Django's template engine auto-escapes all variables by default. This means that any user-supplied data rendered in a template (e.g., customer names, messages, room descriptions) is HTML-escaped to prevent the injection of malicious JavaScript code.
6. **Password Masking:** All password input fields use `type="password"` in the HTML, which masks the characters entered by the user with dots (●), preventing password visibility to bystanders.
7. **OTP-Based Password Reset:** Forgotten passwords are recovered via a time-bound OTP (One-Time Password) sent to the user's registered email address. The OTP is a random 5-digit number stored in the server-side session (not accessible to the client). The user must enter the correct OTP to proceed with password reset, adding an extra layer of security.
8. **Session Management:** User sessions are managed securely by Django's session framework. Session data is stored on the server side. On logout, the session variables are explicitly deleted (`del request.session['USERNAME']`), preventing unauthorized re-access. Django sessions use secure, randomly generated session IDs stored in cookies.
9. **Input Validation:** Server-side validation ensures that mandatory fields are present, numeric fields contain valid numbers, date fields contain valid dates, and email fields contain properly formatted email addresses. This prevents

the storage of corrupt or malicious data in the database.

10. **Clickjacking Protection:** Django's `XFrameOptionsMiddleware` is enabled, which sets the `X-Frame-Options` HTTP header to prevent the application from being embedded in iframes on other websites, protecting against clickjacking attacks.
11. **Secret Key Protection:** The Django `SECRET_KEY` is stored in a `.env` file and loaded using `python-decouple`, ensuring that sensitive configuration is not hardcoded in the source code or version-controlled.

16. Deployment and Installation Guide

This section provides step-by-step instructions for installing and running the Hotel Room Booking application on a development machine.

16.1 Prerequisites

- Python 3.10 or above installed
- pip (Python package manager) installed
- Git installed (optional, for version control)
- A modern web browser (Chrome, Edge, Firefox)
- Internet connection (for payment gateway and email)

16.2 Installation Steps

```
# Step 1: Clone or copy the project folder

cd /path/to/your/directory

# Copy the project folder "Hoteliers_Myproject" here



# Step 2: Create a Python virtual environment
```

```
python -m venv venv



# Step 3: Activate the virtual environment

# On Linux/macOS:

source venv/bin/activate

# On Windows:

venv\Scripts\activate



# Step 4: Install required Python packages

pip install -r requirements.txt



# Step 5: Create a .env file in the project root with:

# SECRET_KEY=your-secret-key-here

# DEBUG=True

# EMAIL_HOST_USER=your-email@gmail.com

# EMAIL_HOST_PASSWORD=your-app-password



# Step 6: Apply database migrations

python manage.py makemigrations
```

```
python manage.py migrate



# Step 7: Create a superuser for admin access

python manage.py createsuperuser



# Step 8: Start the development server

python manage.py runserver



# Step 9: Open browser and navigate to:

# Customer Website: http://127.0.0.1:8000/

# Admin Backend:    http://127.0.0.1:8000/Backend/

# Django Admin:     http://127.0.0.1:8000/admin/
```

16.3 Project Directory Structure

```
Hoteliers_Myproject/

├── manage.py                  # Django management script

├── requirements.txt           # Python package dependencies

├── (PostgreSQL database is used; no file, server-based)
```

```
├── .env                        # Environment variables (SECRET_KEY,
email config)

│

├── Hotelierss/                 # Django project settings

│   ├── settings.py             # Project configuration

│   ├── urls.py                 # Root URL routing

│   ├── wsgi.py                 # WSGI entry point

│   └── asgi.py                 # ASGI entry point

│

├── Backend/                    # Admin backend app

│   ├── models.py               # Room type, room, staff models

│   ├── chatbot_models.py       # Chatbot and notification models

│   ├── views.py                # Admin CRUD views

│   ├── chatbot_views.py        # Chatbot API views

│   ├── urls.py                 # Admin URL routes

│   ├── admin.py                # Django admin registration

│   ├── templates/              # Admin HTML templates

│   ├── static/                 # CSS, JS, images, fonts

│   └── migrations/             # Database migration files
```

```
│
├── webapp/                    # Customer-facing app
│   ├── models.py              # Contact, user, booking, payment
models
│   ├── views.py               # Customer views (booking, payment,
auth)
│   ├── urls.py                # Customer URL routes
│   ├── templates/             # Customer HTML templates
│   └── migrations/            # Database migration files
│
└── media/                     # User-uploaded files
    ├── roomtypeimages/        # Room type images
    ├── roomimage/             # Room images
    └── staffimage/            # Staff photos
```

17. Limitations of the Project

While the system fulfils all the defined objectives, it has the following limitations that should be considered:

1. **Operating System Requirement:** The system requires a modern operating system (Windows 10 or higher, recent Linux distribution, or macOS) capable of running a Python/Django environment. It cannot be run on older systems that do not support Python 3.10+.
2. **Software Dependencies:** The application requires Python, Django, PostgreSQL, and supporting libraries (Razorpay SDK, xhtml2pdf, Pillow, python-decouple) to be installed and properly configured on the server machine. These dependencies must be managed through a virtual environment.
3. **Hardware Requirements:** For smooth performance with multiple concurrent users, the deployment system should have at least a dual-core processor and 4–8 GB RAM. Very low-end hardware may cause slower response times during peak usage.
4. **Rule-Based Chatbot:** The chatbot uses rule-based keyword matching to classify queries and generate responses. It does not support natural language understanding (NLU), sentiment analysis, or machine learning–based response generation. Complex or ambiguous queries may not be understood correctly.
5. **Plain-Text Passwords:** Customer passwords in the custom

Registerdb table are stored as plain text. While Django's built-in User model with hashed passwords (using PBKDF2, Argon2, or bcrypt) would provide stronger security, the current implementation stores passwords as-is for simplicity. This is a known security limitation.

6. **PostgreSQL Database:** The application uses PostgreSQL, which is a server-based relational database suitable for development and production. PostgreSQL supports concurrent write operations efficiently and is recommended for high-traffic production use.
7. **Single-Hotel System:** The system is designed for a single hotel. It does not support multi-hotel or multi-branch management with a centralised dashboard. Each hotel would need a separate installation.
8. **No Dynamic Pricing:** Room prices are fixed values set by the admin. The system does not support dynamic pricing based on demand, season, or occupancy rates.
9. **No Refund Processing:** While the system integrates with Razorpay for payments, it does not currently support automated refund processing for cancelled bookings. Refunds would need to be handled manually through the Razorpay dashboard.
10. **No Mobile App:** The system is a web application only. There is no native mobile app for Android or iOS, although the responsive design ensures a good experience on mobile browsers.

18. Future Scope and Enhancement

This project is designed and developed in such a manner that it provides maximum efficiency and speed, and it has a wide scope for further development. The modular Django architecture allows new features to be added as new apps or modules with minimal changes to the existing database structure and application code. The following enhancements are planned for future versions:

1. **Dynamic Pricing Engine:** Implement an algorithm that automatically adjusts room prices based on demand, season (peak/off-peak), occupancy rates, and competitor pricing. This would maximize revenue and ensure competitive pricing.
2. **Loyalty Program and Discount Coupons:** Build a loyalty program that rewards returning customers with points, discounts, and exclusive offers. Implement a coupon/promo code system that allows customers to apply discount codes during booking.
3. **Multi-Hotel / Multi-Branch Support:** Extend the database schema and admin interface to support multiple hotels or branches under a single platform, with a centralised dashboard for chain management and individual hotel dashboards for branch managers.
4. **Advanced Analytical Reports and Dashboards:** Build interactive dashboards with charts and graphs for occupancy rates, revenue trends, customer demographics, booking patterns, staff performance, and chatbot usage analytics using libraries like Chart.js or Plotly.

5. **AI/NLP-Based Chatbot:** Replace the rule-based keyword matching with a Natural Language Processing (NLP) engine using machine learning models (such as those from Hugging Face Transformers or OpenAI API) for deeper conversational understanding, context retention, and more natural responses.
6. **OTA Channel Integration:** Integrate with external hotel booking channels (Online Travel Agencies) such as Booking.com, MakeMyTrip, and Goibibo APIs to synchronise room availability and bookings across multiple platforms.
7. **Advanced Payment Features:** Implement refund processing, invoice generation (with GST calculations), multiple payment methods (wallet, EMI), and payment history tracking with transaction IDs.
8. **Customer Review and Rating System:** Allow customers to leave reviews and ratings for rooms, services, and their overall stay experience. Display aggregate ratings on room pages to help future customers make informed decisions.
9. **SMS Notifications:** Add SMS-based notifications (using Twilio or MSG91 API) in addition to email alerts for booking confirmations, OTPs, check-in reminders, and promotional messages.
10. **Production Database:** PostgreSQL is used for production-grade scalability, concurrent write support, and cloud deployment readiness.
11. **Cloud Deployment:** Deploy the application on cloud platforms such as AWS (Elastic Beanstalk), Google Cloud (App Engine), DigitalOcean, or Heroku for high availability, scalability, and global access.
12. **Native Mobile Application:** Develop a companion mobile app for Android and iOS using React Native or Flutter that

communicates with the Django backend via REST APIs.

13. **Multi-Language Support:** Implement internationalization (i18n) using Django's built-in translation framework to support multiple languages for global customers.
14. **Hashed Password Storage:** Migrate from plain-text password storage to Django's built-in User model with PBKDF2/Argon2 password hashing for stronger security compliance.
15. **Room Amenity Tagging:** Add a tagging system for room amenities (Wi-Fi, AC, TV, Mini-bar, Balcony, etc.) and allow customers to filter rooms by specific amenities during search.

19. Glossary of Terms

Term	Definition
AJAX	Asynchronous JavaScript and XML – A technique for sending/receiving data from a server without reloading the full web page.
API	Application Programming Interface – A set of defined rules and protocols for building and interacting with software applications.
Bootstrap	A popular open-source CSS framework for building responsive, mobile-first web interfaces.
CRUD	Create, Read, Update, Delete – The four basic operations performed on database records.
CSRF	Cross-Site Request Forgery – A web security vulnerability where an attacker tricks a user into performing unintended actions.
DFD	Data Flow Diagram – A graphical representation of data flow through a system, showing processes, data

	stores, and external entities.
Django	A high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern.
ER Diagram	Entity-Relationship Diagram – A visual representation of entities, attributes, and their relationships in a database.
FK (Foreign Key)	A field in a database table that references the primary key of another table, establishing a relationship.
JSON	JavaScript Object Notation – A lightweight data interchange format used for API communication.
MVT	Model-View-Template – Django's architectural pattern where Models define data, Views handle logic, and Templates render HTML.
NLP	Natural Language Processing – A branch of AI that deals with understanding and processing human language.
ORM	Object-Relational Mapper – A tool that maps database tables to Python classes and SQL queries to Python

	method calls.
OTA	Online Travel Agency – A website that allows consumers to book travel products (e.g., Booking.com, MakeMyTrip).
OTP	One-Time Password – A temporary password sent via email or SMS for authentication purposes.
PK (Primary Key)	A unique identifier for each record in a database table.
RDBMS	Relational Database Management System – A database system that stores data in tables with relationships between them.
REST API	Representational State Transfer API – An architectural style for designing web services that use HTTP methods.
Razorpay	An Indian payment gateway provider that enables online payments via cards, UPI, net banking, and wallets.
SDLC	Software Development Life Cycle – A structured process for developing software through defined

	phases.
PostgreSQL	An advanced, open-source, server-based relational database engine suitable for production and development.
SRS	Software Requirement Specification – A document that describes all the requirements of a software system.
UI	User Interface – The visual elements through which a user interacts with a software application.
XSS	Cross-Site Scripting – A web security vulnerability where malicious scripts are injected into web pages.
xhtml2pdf	A Python library that converts HTML/CSS documents to PDF format.

20. Bibliography

20.1 Websites and Online Documentation

1. **Django Official Documentation**
   https://docs.djangoproject.com/
   Comprehensive documentation covering Django models, views, templates, forms, authentication, ORM, middleware, and deployment. Used extensively throughout development.
2. **Python Official Documentation**
   https://www.python.org/doc/
   Official Python language reference, standard library documentation, and tutorials. Referenced for Python syntax, data types, and built-in functions.
3. **PostgreSQL Documentation**
   https://www.postgresql.org/docs/
   Reference for SQL syntax, database design, normalization, and query optimization. Used for understanding RDBMS concepts applicable to the project.
4. **Razorpay Django Integration Guide**
   https://razorpay.com/docs/payments/server-integration/django/
   Official Razorpay documentation for integrating the payment gateway with Django applications. Used for order creation, payment capture, and SDK setup.
5. **Bootstrap 5 Documentation**

https://getbootstrap.com/docs/
Documentation for Bootstrap's grid system, components (cards, forms, navbars, modals), utilities, and responsive breakpoints. Used for all frontend design.

6. **MDN Web Docs – JavaScript Reference**
   https://developer.mozilla.org/en-US/docs/Web/JavaScript
   Mozilla Developer Network reference for JavaScript, DOM manipulation, Fetch API, and event handling. Used for AJAX search, chatbot communication, and dynamic UI updates.
7. **xhtml2pdf Documentation**
   https://xhtml2pdf.readthedocs.io/
   Documentation for the xhtml2pdf library used to convert HTML templates to downloadable PDF documents.
8. **Stack Overflow**
   https://stackoverflow.com/
   Community Q&A platform used for troubleshooting specific coding issues related to Django, Python, JavaScript, and database queries.
9. **W3Schools**
   https://www.w3schools.com/
   Online tutorials and references for HTML, CSS, JavaScript, SQL, and Python. Used for quick syntax lookups and code examples.

20.2 Reference Books – Python, Django & Web Development

1. Adrian Holovaty, Jacob Kaplan-Moss – *The Definitive Guide to Django: Web Development Done Right*, 2nd Edition, Apress.

Comprehensive guide to Django framework covering models, views, templates, forms, admin, and deployment.

2. William S. Vincent – *Django for Beginners: Build Websites with Python & Django*, WelcomeToCode.
   Beginner-friendly guide to building web applications with Django, covering project setup, CRUD operations, user authentication, and deployment.
3. Eric Matthes – *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*, 3rd Edition, No Starch Press.
   Introduction to Python programming with projects including web applications using Django.
4. Robin Nixon – *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5*, 6th Edition, O'Reilly Media.
   Reference for general web development concepts, client-server architecture, and database interaction patterns.
5. Antonio Melé – *Django 4 by Example*, 4th Edition, Packt Publishing.
   Practical guide with real-world Django projects covering e-commerce, social networks, and REST APIs.

20.3 Reference Books – Databases & SQL

1. Regina O. Obe, Leo S. Hsu – *PostgreSQL: Up and Running*, 3rd Edition, O'Reilly Media.
   Reference for relational database concepts, SQL queries, indexing, and database administration.
2. Allen G. Taylor – *SQL For Dummies*, 9th Edition, John Wiley & Sons.
   Beginner-friendly introduction to SQL, covering SELECT,

INSERT, UPDATE, DELETE, JOINs, and subqueries.

3. Abraham Silberschatz, Henry F. Korth, S. Sudarshan – *Database System Concepts*, 7th Edition, McGraw-Hill Education.
   Textbook covering relational model, normalization, ER diagrams, SQL, transaction management, and database design theory.

20.4 IGNOU Study Materials

1. IGNOU – *BCS-051: Introduction to Software Engineering*
   Covered SDLC, SRS, DFD, ER diagrams, testing strategies, and project documentation standards.
2. IGNOU – *BCS-053: Web Programming*
   Covered HTML, CSS, JavaScript, server-side programming, and database connectivity.
3. IGNOU – *BCS-054: Computer Oriented Numerical Techniques*
   Covered algorithmic thinking and computational methods applicable to software development.
4. IGNOU – *BCSL-057: Web Programming Lab*
   Practical exercises in web development using HTML, CSS, JavaScript, and server-side technologies.
5. IGNOU – *BCSP-064: BCA Project Guidelines*
   Official project guidelines covering report structure, formatting, submission requirements, and evaluation criteria.

Checklist before submission:

☐ Fill in blank fields: Study Centre Code, Guide Name, Designation,

Qualification, Address

☐ Attach original screenshots of all 26 input/output screens

☐ Attach the approved Project Proposal Proforma

☐ Attach the approved Synopsis

☐ Attach the Guide's Bio-Data

☐ Get the report hard-bound (as per IGNOU guidelines)

☐ Submit 2 copies: 1 to the Study Centre, 1 to the Regional Centre

☐ Keep 1 copy for your own records